



4장 분류 Part2

4팀 진규빈, 노현선, 김두현

목차

#01 GBM

#02 XGBoost

#03 LightGBM

#04 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝

#05 분류 실습

#06 스택킹 앙상블



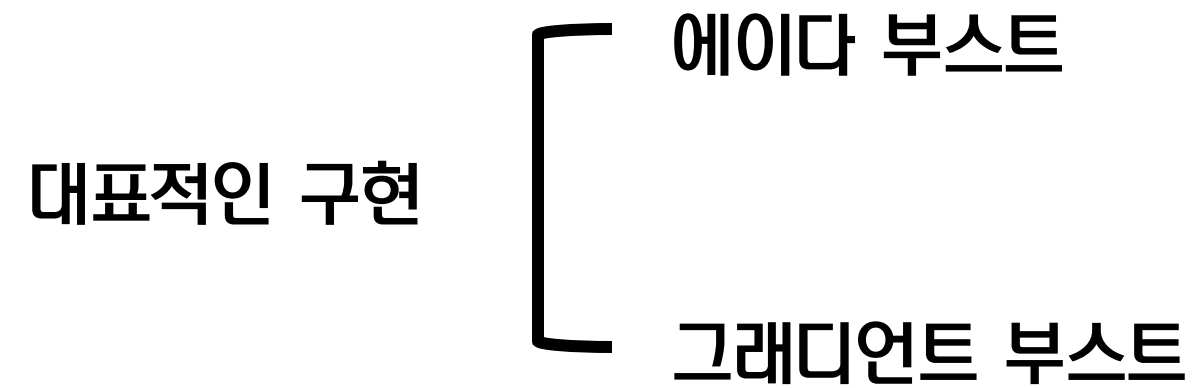
5. GBM



GBM의 개요 및 실습

부스팅 알고리즘

여러 개의 약한 학습기(weak learner)를 순차적으로 학습-예측하면서
잘못 학습한 데이터에 가중치 부여를 통해 오류를 개선해 나가면서 학습하는 방식



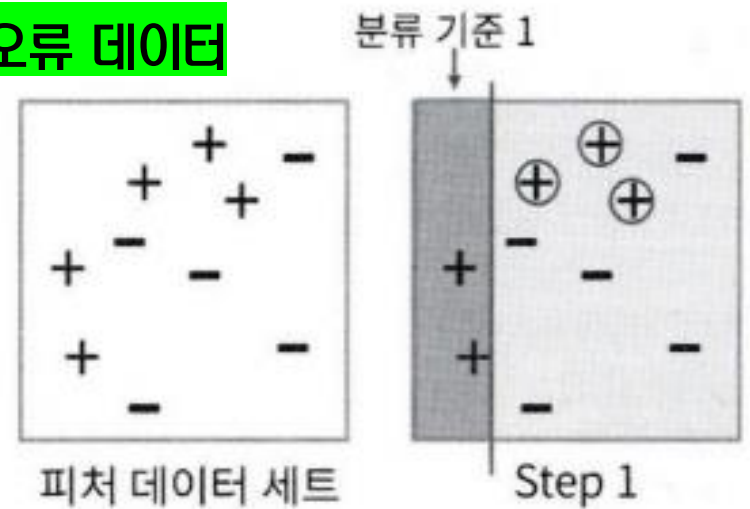
GBM의 개요 및 실습

에이다 부스트 (AdaBoost = Adaptive Boosting)

오류 데이터에 가중치 부여하면서 부스팅 수행하는 알고리즘

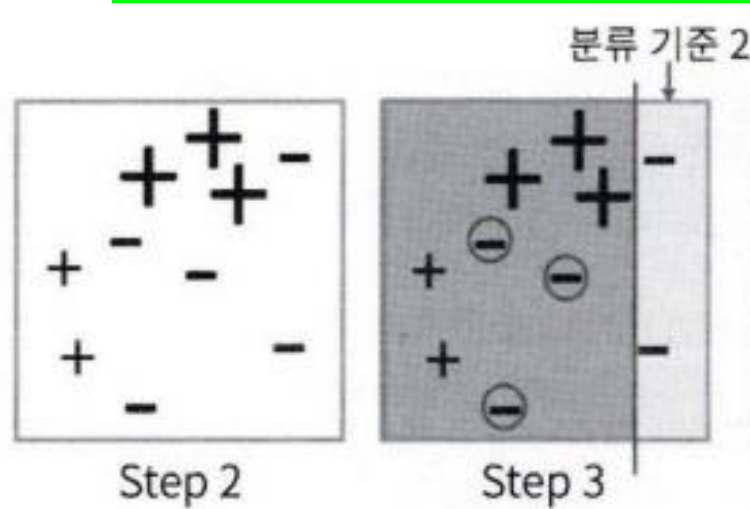
Step 1) 첫 번째 약한 학습기가 분류 기준 1로 +와 - 분류

⊕ 데이터가 잘못 분류된 오류 데이터



Step 3) 두 번째 약한 학습기가 분류 기준 2로 +와 - 분류

⊖ 데이터가 잘못 분류된 오류 데이터

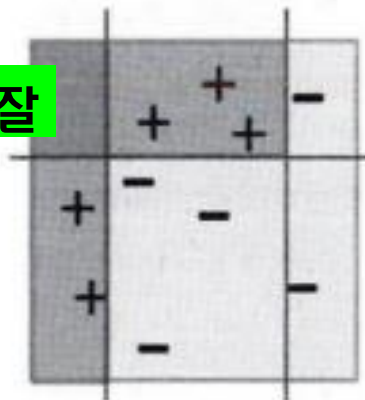


Step 2) 오류 데이터에 대해 가중치 값 부여

가중치 부여된 오류 ⊕ 데이터는 다음 약한 학습기가 더 잘 분류할 수 있게 크기 커짐

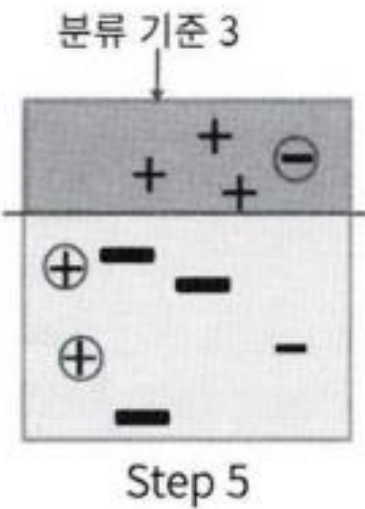
Step 4) 오류 ⊖ 데이터에 대해 더 큰 가중치 부여

오류 ⊖ 데이터의 크기 커짐



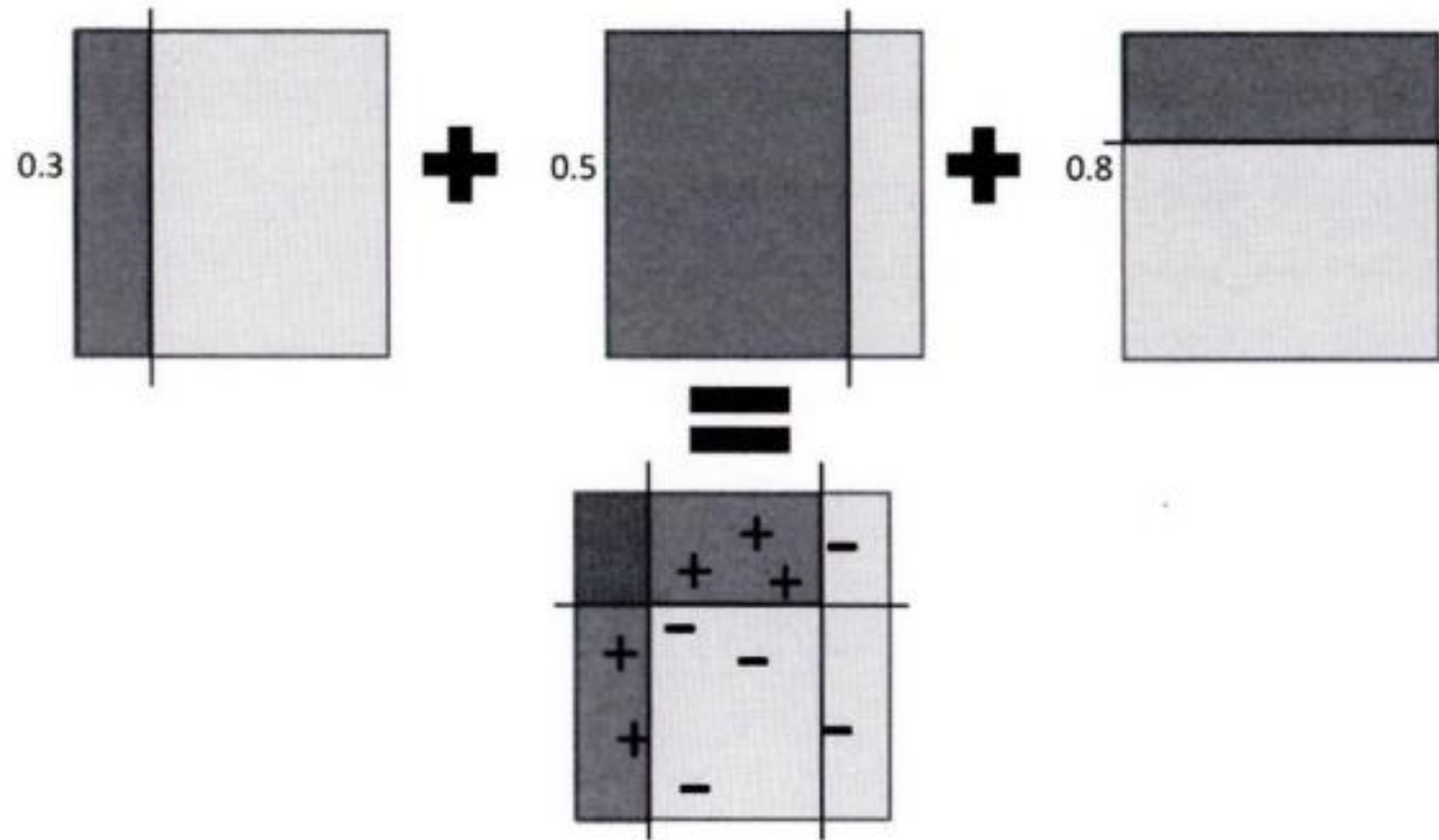
개별 약한 학습기보다 훨씬 정확도 높아짐

Step 5) 세 번째 약한 학습기가 분류 기준 3으로 +와 - 분류하고 오류 데이터 찾음



GBM의 개요 및 실습

ex)



GBM의 개요 및 실습

GBM (Gradient Boosting Machine)

에이다 부스트와 유사하지만
가중치 업데이트에 경사 하강법 이용하는 것이 차이
오류 값 = 실제 값 - 예측값

[오류식]

$$h(x) = y - F(x)$$

y: 실제 결과값

$x_1, x_2, \dots, x_n$: 피쳐

$F(x)$: 예측 함수

경사 하강법

이 오류식 최소화하는 방향성 가지고
반복적으로 가중치 값 업데이트하는 것

GBM의 개요 및 실습

GradientBoostingClassifier

[10]
✓ 23
분

```
from sklearn.ensemble import GradientBoostingClassifier
import time
import warnings
warnings.filterwarnings('ignore')

X_train,X_test,y_train,y_test=get_human_dataset()

# GBM 수행 시간 측정을 위함. 시작 시간 설정
start_time=time.time()

gb_clf=GradientBoostingClassifier(random_state=0)
gb_clf.fit(X_train,y_train)
gb_pred=gb_clf.predict(X_test)
gb_accuracy=accuracy_score(y_test,gb_pred)

print('GBM 정확도:{0:.4f}'.format(gb_accuracy))
print('GBM 수행 시간:{0:.1f}초'.format(time.time()-start_time))
```

⇒ GBM 정확도:0.9379
GBM 수행 시간:1426.6초

약한 학습기의 순차적인 예측 오류 보정 통해 학습 수행하므로
멀티 CPU 코어 시스템 사용하더라도 병렬 처리 지원 X
=> 대용량 데이터 학습에 매우 많은 시간 필요

일반적으로 랜덤포레스트보다 조금 더 나은 예측 성능
But 수행 시간 오래 걸림 & 하이퍼 파라미터 튜닝 노력 더 필요

GBM 하이퍼 파라미터 소개

loss	경사 하강법에서 사용할 비용 함수 지정 기본값: deviance
learning_rate	GBM이 학습 진행할 때마다 적용하는 학습률 Weak learner가 순차적으로 오류 값 보정해 나가는 데 적용하는 계수 0~1 사이 값 지정 / 기본값: 0.1
n_estimators	Weak learner의 개수 기본값: 100
subsample	Weak learner가 학습에 사용하는 데이터의 샘플링 비율 기본값: 1 / 과적합 염려되는 경우 1보다 작은 값으로 설정

6. XGBoost



XGBoost 개요

XGBoost (eXtra Gradient Boost)

- 트리 기반의 앙상블 학습에서 가장 각광받고 있는 알고리즘 중 하나
- 분류에 있어서 일반적으로 다른 머신러닝보다 뛰어난 예측 성능 나타냄
- GBM에 기반하고 있지만, GBM의 단점인 느린 수행 시간, 과적합 규제 부재 등의 문제 해결
- 병렬 CPU 환경에서 병렬 학습 가능 => 기존 GBM보다 빠르게 학습 완료

XGBoost 개요

장점

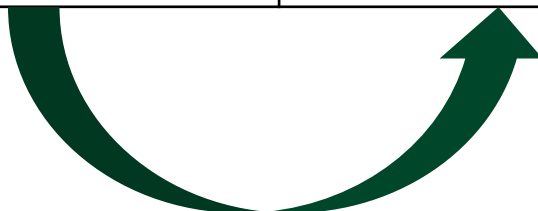
뛰어난 예측 성능	일반적으로 분류 & 회귀 영역에서 예측 성능 뛰어남
GBM 대비 빠른 수행 시간	일반적인 GBM은 순차적으로 Weak learner가 가중치 증감하는 방식으로 학습하여 전반적 속도 느림 <=> XGBoost는 병렬 수행 및 다양한 기능으로 GBM보다 빠른 수행 성능 보장 (다른 머신러닝 알고리즘에 비해 빠르다는 의미는 아님)
과적합 규제 (Regularization)	표준 GBM에는 과적합 규제 기능 없음 XGBoost는 자체에 과적합 규제 기능 있어 과적합에 더 강한 내구성 가짐
나무 가지치기 (Tree Pruning)	일반적으로 GBM은 분할 시 부정 손실 발생하면 분할 더 이상 수행하지 않음 => 지나치게 많은 분할 발생할 수 있음 XGBoost도 max_depth 파라미터로 분할 깊이 조정하기도 하지만 나무 가지치기로 더 이상 긍정이득 없는 분할을 가지치기 해 분할 수 더 줄이는 추가적인 장점 가짐
자체 내장된 교차 검증	XGBoost는 반복 수행 시마다 내부적으로 학습&평가 데이터 세트에 대한 교차 검증 수행해 최적화된 반복 수행 횟수 가짐 조기 중단 기능(지정된 반복 횟수 아니라 교차 검증 통해 평가 데이터 세트의 평가 값이 최적화 되면 반복 중간에 멈출 수 있음)
결손값 자체 처리	XGBoost는 결손값 자체 처리할 수 있는 기능 가짐

XGBoost 개요

XGBoost (eXtra Gradient Boost)

- 핵심 라이브러리는 C/C++로 작성
- 파이썬 패키지 "xgboost" 제공 => C/C++ 핵심 라이브러리 호출하도록
- XGBoost 전용의 파이썬 패키지 & 사이킷런과 호환되는 래퍼용 XGBoost 함께 존재

XGBoost 전용의 파이썬 패키지	사이킷런과 호환되는 래퍼용 XGBoost
- 초기 출시 당시 사이킷런과 호환 X - XGBoost 고유의 프레임워크를 파이썬 언어 기반에서 구현한 것 / 독자적인, 별도의 API 기반 - fit(), predict() 메서드 등 사이킷런 고유 아키텍처 적용 X - cross_val_score, GridSearchCV, Pipeline 등 다양한 유틸리티 사용 X	- 사이킷런과 연동할 수 있는 래퍼 클래스 - XGBClassifier & XGBRegressor - 사이킷런의 다른 Estimator와 사용법 같음



파이썬 래퍼 XGBoost 하이퍼 파라미터

XGBoost 하이퍼 파라미터

= GBM 하이퍼 파라미터 + 조기 중단(early stopping) + 과적합 규제 하이퍼 파라미터

일반 파라미터

일반적으로 실행 시 스레드의 개수나 silent 모드 등의 선택 위한 파라미터
(디폴트 파라미터 값 바꾸는 경우 거의 X)

- booster: gbtree(tree based model) 또는 gblinear(linear model) 선택. 디폴트는 gbtree입니다.
- silent: 디폴트는 0이며, 출력 메시지를 나타내고 싶지 않을 경우 1로 설정합니다.
- nthread: CPU의 실행 스레드 개수를 조정하며, 디폴트는 CPU의 전체 스레드를 다 사용하는 것입니다. 멀티 코어/스레드 CPU 시스템에서 전체 CPU를 사용하지 않고 일부 CPU만 사용해 ML 애플리케이션을 구동하는 경우에 변경합니다.

파이썬 래퍼 XGBoost 하이퍼 파라미터

부스터 파라미터

트리 최적화, 부스팅, regularization 등과 관련 파라미터 등

- eta [default=0.3, alias: learning_rate]: GBM의 학습률(learning rate)과 같은 파라미터입니다. 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값. 파이썬 래퍼 기반의 xgboost를 이용할 경우 디폴트는 0.3. 사이킷런 래퍼 클래스를 이용할 경우 eta는 learning_rate 파라미터로 대체되며, 디폴트는 0.1입니다. 보통은 0.01 ~ 0.2 사이의 값을 선호합니다.
- num_boost_rounds: GBM의 n_estimators와 같은 파라미터입니다.
- min_child_weight [default=1]: 트리에서 추가적으로 가지를 나눌지를 결정하기 위해 필요한 데이터들의 weight 총합. min_child_weight이 클수록 분할을 자제합니다. 과적합을 조절하기 위해 사용됩니다.
- gamma [default=0, alias: min_split_loss]: 트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값입니다. 해당 값보다 큰 손실(loss)이 감소된 경우에 리프 노드를 분리합니다. 값이 클수록 과적합 감소 효과가 있습니다.
- max_depth [default=6]: 트리 기반 알고리즘의 max_depth와 같습니다. 0을 지정하면 깊이에 제한이 없습니다. Max_depth가 높으면 특정 피쳐 조건에 특화되어 룰 조건이 만들어지므로 과적합 가능성이 높아지며 보통은 3~10 사이의 값을 적용합니다.
- sub_sample [default=1]: GBM의 subsample과 동일합니다. 트리가 커져서 과적합되는 것을 제어하기 위해 데이터를 샘플링하는 비율을 지정합니다. sub_sample=0.5로 지정하면 전체 데이터의 절반을 트리를 생성하는 데 사용합니다. 0에서 1 사이의 값이 가능하나 일반적으로 0.5 ~ 1 사이의 값을 사용합니다.
- colsample_bytree [default=1]: GBM의 max_features와 유사합니다. 트리 생성에 필요한 피쳐(칼럼)를 임의로 샘플링하는 데 사용됩니다. 매우 많은 피쳐가 있는 경우 과적합을 조정하는 데 적용합니다.
- lambda [default=1, alias: reg_lambda]: L2 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.
- alpha [default=0, alias: reg_alpha]: L1 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.
- scale_pos_weight [default=1]: 특정 값으로 치우친 비대칭한 클래스로 구성된 데이터 세트의 균형을 유지하기 위한 파라미터입니다.

파이썬 래퍼 XGBoost 하이퍼 파라미터

학습 태스크 파라미터 학습 수행 시의 객체 함수, 평가 위한 지표 등 설정하는 파라미터

- **objective:** 최솟값을 가져야 할 손실 함수를 정의합니다. XGBoost는 많은 유형의 손실함수를 사용할 수 있습니다. 주로 사용되는 손실함수는 이진 분류인지 다중 분류인지에 따라 달라집니다.
- **binary:logistic:** 이진 분류일 때 적용합니다.
- **multi:softmax:** 다중 분류일 때 적용합니다. 손실함수가 multi:softmax일 경우에는 레이블 클래스의 개수인 num_class 파라미터를 지정해야 합니다.
- **multi:softprob:** multi:softmax와 유사하나 개별 레이블 클래스의 해당되는 예측 확률을 반환합니다.
- **eval_metric:** 검증에 사용되는 함수를 정의합니다. 기본값은 회귀인 경우는 rmse, 분류일 경우에는 error입니다. 다음은 eval_metric의 값 유형입니다.
 - rmse: Root Mean Square Error
 - mae: Mean Absolute Error
 - logloss: Negative log-likelihood
 - error: Binary classification error rate (0.5 threshold)
 - merror: Multiclass classification error rate
 - mlogloss: Multiclass logloss
 - auc: Area under the curve

파이썬 래퍼 XGBoost 하이퍼 파라미터

파라미터 튜닝 - 과적합 문제 심각할 경우 적용할 만한 것

eta 값 낮추기(0.01~0.1)
=> num_round(또는 n_estimators)는 반대로 높여줘야 함

max_depth 값 낮추기

min_child_weight 값 높이기

gamma 값 높이기

Subsample & colsample_bytree 조정
=> 트리가 너무 복잡하게 생성되는 것 막음

파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

Xgboost 모듈 로딩하고 xgb로 명명

plot_importance: 피처의 중요도 시각화해주는 모듈

```
[2] 0초
import xgboost as xgb
from xgboost import plot_importance
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

dataset=load_breast_cancer()
features=dataset.data
labels=dataset.target
cancer_df=pd.DataFrame(data=features,columns=dataset.feature_names)
cancer_df['target']=labels
cancer_df.head(3)
```

사이킷런에 내장된 위스콘신 유방암 데이터 세트
load_breast_cancer()로 호출

타깃 레이블 값 종류
악성(malignant) => 0 / 양성(benign) => 1

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst cor pc
0	17.99	10.38	122.8	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.6	2019.0	0.1622	0.6656	0.7119	0
1	20.57	17.77	132.9	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.8	1956.0	0.1238	0.1866	0.2416	0
2	19.69	21.25	130.0	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.5	1709.0	0.1444	0.4245	0.4504	0

3 rows × 31 columns

파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

[3]
✓ 0초

```
▶ print(dataset.target_names)  
print(cancer_df['target'].value_counts())
```

```
⇒ ['malignant' 'benign']  
target  
1      357  
0      212  
Name: count, dtype: int64
```

1 값 – 양성(benign) - 357개
0 값 – 악성(malignant) - 212개

파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

데이터 세트의 80%를 학습용으로, 20%를 테스트용으로 추출
⇒ 80%의 학습용 데이터에서 90%를 최종 학습용, 10%를 검증용으로 분할
(검증 성능 평가 & 조기 중단 수행해 보기 위해)

[4]

✓ 0초

```
# cancer_df에서 feature용 DataFrame과 Label용 Series 객체 추출
# 맨 마지막 칼럼이 Label임. Feature용 DataFrame은 cancer_df의 첫번째 칼럼에서 맨 마지막 두번째 칼럼까지를 :-1 슬라이싱으로 추출
X_features=cancer_df.iloc[:, :-1]
y_label=cancer_df.iloc[:, -1]

# 전체 데이터 중 80%는 학습용 데이터, 20%는 테스트용 데이터 추출
X_train,X_test,y_train,y_test=train_test_split(X_features,y_label,test_size=0.2,random_state=156)

# 위에서 만든 X_train, y_train을 다시 쪼개서 90%는 학습과 10%는 검증용 데이터로 분리
X_tr,X_val,y_tr,y_val=train_test_split(X_train,y_train,test_size=0.1,random_state=156)

print(X_train.shape,X_test.shape)
print(X_tr.shape,X_val.shape)
```

⇒ (455, 30) (114, 30)
(409, 30) (46, 30)

전체 569개 데이터 세트에서 최종 학습용 409개, 검증용 46개, 테스트용 114개 추출

파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

Dmatrix: XGBoost만의 전용 데이터 객체

- Numpy 또는 Pandas로 되어 있는 학습용, 검증, 테스트용 데이터 세트 모두 DMatrix로 생성하여 모델에 입력해 줘야 함
- 넘파이, DataFrame, Series 외에 libsvm txt 포맷 파일, xgboost 이진 버퍼 파일을 파라미터로 입력받아 변환할 수 있음
- 주요 입력 파라미터: data, label
 - * data => 피쳐 데이터 세트
 - * label => (분류) 레이블 데이터 세트 / (회귀) 종속값 데이터 세트

[4]
✓ 6초



```
# 만약 구버전 XGBoost에서 DataFrame으로 DMatrix 생성이 안 될 경우 X_train.values로 넘파이 변환
# 학습, 검증, 테스트용 DMatrix 생성
dtr = xgb.DMatrix(data=X_tr.values, label=y_tr.values)
dval = xgb.DMatrix(data=X_val.values, label=y_val.values)
dtest = xgb.DMatrix(data=X_test.values, label=y_test.values)
```


파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

XGBoost의 하이퍼 파라미터는 주로 **딕셔너리** 형태로 입력

- max_depth(트리 최대 깊이)는 3.
- 학습률 eta는 0.1(XGBClassifier를 사용할 경우 eta가 아니라 learning_rate입니다).
- 예제 데이터가 0 또는 1 이진 분류이므로 목적함수(objective)는 이진 로지스틱(binary:logistic).
- 오류 함수의 평가 성능 지표는 logloss.
- num_rounds(부스팅 반복 횟수)는 400회

[5]

✓ 0초

```
params={'max_depth':3,'eta':0.05,'objective':'binary:logistic','eval_metric':'logloss'}  
num_rounds=400
```

파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

지정된 파라미터로 XGBoost 모델 학습

파이썬 래퍼 XGBoost는 하이퍼 파라미터를 xgboost 모듈의 train() 함수에 파라미터로 전달

[6]
✓ 2초

```
# 학습 데이터 셋은 'train' 또는 평가 데이터 셋은 'eval'로 명기합니다
eval_list=[(dtr, 'train'), (dval, 'eval')] # 또는 eval_list=[(dval, 'eval')]만 명기해도 무방

# 하이퍼 파라미터와 early stopping 파라미터를 train() 함수의 파라미터로 전달
xgb_model=xgb.train(params, dtrain=dtr, num_boost_round=num_rounds, early_stopping_rounds=50, evals=eval_list)
```

```
[118] train-logloss:0.02176 eval-logloss:0.25728
[119] train-logloss:0.02149 eval-logloss:0.25722
[120] train-logloss:0.02119 eval-logloss:0.25764
[121] train-logloss:0.02095 eval-logloss:0.25761
[122] train-logloss:0.02067 eval-logloss:0.25832
[123] train-logloss:0.02045 eval-logloss:0.25808
[124] train-logloss:0.02023 eval-logloss:0.25855
[125] train-logloss:0.01998 eval-logloss:0.25714
[126] train-logloss:0.01973 eval-logloss:0.25587
[127] train-logloss:0.01946 eval-logloss:0.25640
[128] train-logloss:0.01927 eval-logloss:0.25685
[129] train-logloss:0.01908 eval-logloss:0.25665
[130] train-logloss:0.01886 eval-logloss:0.25712
[131] train-logloss:0.01863 eval-logloss:0.25609
[132] train-logloss:0.01839 eval-logloss:0.25649
[133] train-logloss:0.01816 eval-logloss:0.25789
[134] train-logloss:0.01802 eval-logloss:0.25811
[135] train-logloss:0.01785 eval-logloss:0.25794
[136] train-logloss:0.01763 eval-logloss:0.25876
[137] train-logloss:0.01748 eval-logloss:0.25884
[138] train-logloss:0.01732 eval-logloss:0.25867
[139] train-logloss:0.01719 eval-logloss:0.25876
[140] train-logloss:0.01696 eval-logloss:0.25987
```

조기 중단

XGBoost가 수행 성능 개선 위해 더 이상 지표 개선 없을 경우
num_boost_round 수행 횟수 모두 채우지 않고 중간에 반복 빠져 나올
수 있도록 하는 것

* 주로 별도의 검증 데이터 세트 이용해 성능 평가

train() 함수에 early_stopping_rounds 파라미터 입력!
이때 반드시 평가용 데이터 세트 지정 & eval_metric 함께 설정
eval_metric => 평가 세트에 적용할 성능 평가 방법
(분류일 경우 주로 'error', 'logloss' 적용)

파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

학습용 DMatrix인 dtr과 검증용 DMatrix인 dval로
설정된 뒤 train() 함수의 evals 인자값으로 입력

[6]
✓ 2초

```
# 학습 데이터 셋은 'train' 또는 평가 데이터 셋은 'eval'로 명기합니다
eval_list=[(dtr, 'train'), (dval, 'eval')] # 또는 eval_list=[(dval, 'eval')]만 명기해도 무방

# 하이퍼 파라미터와 early stopping 파라미터를 train() 함수의 파라미터로 전달
xgb_model=xgb.train(params, dtrain=dtr, num_boost_round=num_rounds, early_stopping_rounds=50, evals=eval_list)
```

[118]	train-logloss:0.02176	eval-logloss:0.25728
[119]	train-logloss:0.02149	eval-logloss:0.25722
[120]	train-logloss:0.02119	eval-logloss:0.25764
[121]	train-logloss:0.02095	eval-logloss:0.25761
[122]	train-logloss:0.02067	eval-logloss:0.25832
[123]	train-logloss:0.02045	eval-logloss:0.25808
[124]	train-logloss:0.02023	eval-logloss:0.25855
[125]	train-logloss:0.01998	eval-logloss:0.25714
[126]	train-logloss:0.01973	eval-logloss:0.25587
[127]	train-logloss:0.01946	eval-logloss:0.25640
[128]	train-logloss:0.01927	eval-logloss:0.25685
[129]	train-logloss:0.01908	eval-logloss:0.25665
[130]	train-logloss:0.01886	eval-logloss:0.25712
[131]	train-logloss:0.01863	eval-logloss:0.25609
[132]	train-logloss:0.01839	eval-logloss:0.25649
[133]	train-logloss:0.01816	eval-logloss:0.25789
[134]	train-logloss:0.01802	eval-logloss:0.25811
[135]	train-logloss:0.01785	eval-logloss:0.25794
[136]	train-logloss:0.01763	eval-logloss:0.25876
[137]	train-logloss:0.01748	eval-logloss:0.25884
[138]	train-logloss:0.01732	eval-logloss:0.25867
[139]	train-logloss:0.01719	eval-logloss:0.25876
[140]	train-logloss:0.01696	eval-logloss:0.25987

XGBoost 학습 반복 시마다 evals에 설정된 데이터 세트에 대해 평가
지표 결과 출력, train()은 학습 완료된 모델 객체 반환

반복 시마다 train-logloss와 eval-logloss가 지속적으로 감소

파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

[7]
✓ 0초



```
pred_probs=xgb_model.predict(dtest)
print('predict() 수행 결과값을 10개만 표시, 예측 확률 값으로 표시됨')
print(np.round(pred_probs[:10],3))
```

```
# 예측 확률이 0.5보다 크면 1, 그렇지 않으면 0으로 예측값 결정하여 List 객체인 preds에 저장
preds=[1 if x>0.5 else 0 for x in pred_probs]
print('예측값 10개만 표시:',preds[:10])
```



```
predict() 수행 결과값을 10개만 표시, 예측 확률 값으로 표시됨
[0.845 0.008 0.68  0.081 0.975 0.999 0.998 0.998 0.996 0.001]
예측값 10개만 표시: [1, 0, 1, 0, 1, 1, 1, 1, 1, 0]
```

지정된 파라미터로 XGBoost 모델 학습

파이썬 래퍼 XGBoost는 하이퍼 파라미터를 xgboost 모듈의 train() 함수에 파라미터로 전달

파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

[8]

✓ 0초



```
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import precision_score, recall_score
from sklearn.metrics import f1_score, roc_auc_score
```

```
def get_clf_eval(y_test, pred=None, pred_proba=None):
    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)
    # ROC-AUC 추가
    roc_auc = roc_auc_score(y_test, pred_proba)
    print('오차 행렬')
    print(confusion)
    # ROC-AUC print 추가
    print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f}, F1: {3:.4f}, AUC: {4:.4f}'.format(accuracy, precision, recall, f1, roc_auc))
```

3장에서 생성한 get_clf_eval() 함수 적용해
XGBoost 모델의 예측 성능 평가

[9]

✓ 0초



```
get_clf_eval(y_test, preds, pred_probs)
```

```
오차 행렬
[[34  3]
 [ 2 75]]
```

```
정확도: 0.9561, 정밀도: 0.9615, 재현율: 0.9740, F1: 0.9677, AUC:0.9937
```

테스트 실제 레이블 값 가지는 y_test와 예측 레이블인 preds, 예측 확률인 pred_proba를 인자로 입력

파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

xgboost 패키지에 내장된 시각화 기능 수행

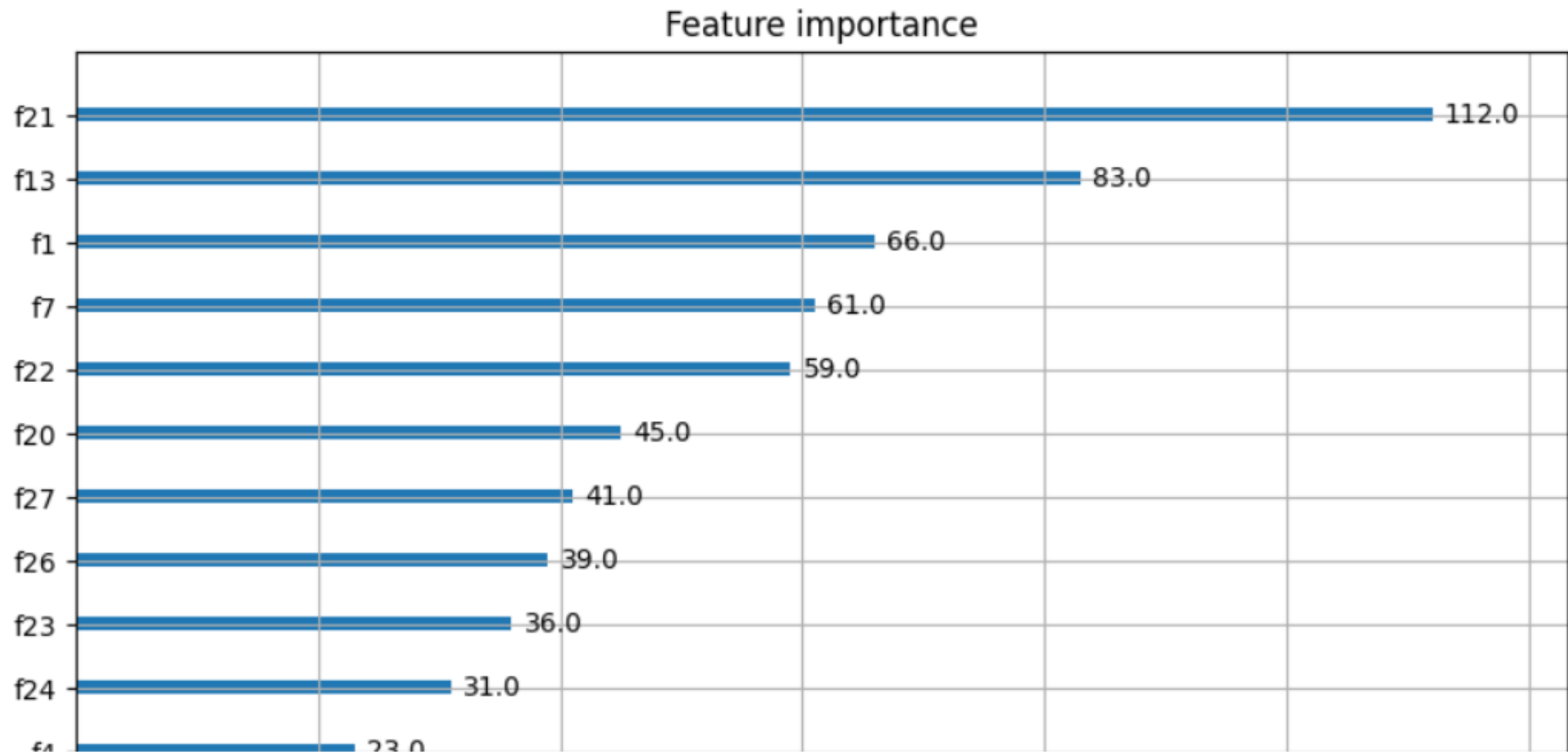
plot_importance() API => 피처의 중요도를 막대그래프 형식으로 나타냄
파라미터로 앞에 학습 완료된 모델 객체 및 맷플롯립의 ax 객체 입력

f스코어 기반으로 해당 피처의 중요도 나타냄
f스코어: 해당 피처가 트리 분할 시 얼마나 자주 사용되었는지 지표로 나타낸 값

```
[10]
✓ 0초
import matplotlib.pyplot as plt
%matplotlib inline

fig,ax=plt.subplots(figsize=(10,12))
plot_importance(xgb_model,ax=ax)
```

<Axes: title={'center': 'Feature importance'}, xlabel='F score', ylabel='Features'>



Y축의 피처명 나열 시
f0, f1과 같이 f자 뒤에
순서 붙여 피처명 나타냄
(f0: 첫 번째 피처, f1: 두 번째 피처)

파이썬 래퍼 XGBoost 적용 – 위스콘신 유방암 예측

파이썬 래퍼 XGBoost는 사이킷런의 GridSearchCV와 유사하게 데이터 세트에 대한 교차 검증 수행 후 최적 파라미터 구할 수 있는 방법을 `cv()` API로 제공

* `xgb.cv`의 반환값은 DataFrame 형태

```
xgboost.cv(params, dtrain, num_boost_round=10, nfold=3, stratified=False, folds=None, metrics=(),  
obj=None, feval=None, maximize=False, early_stopping_rounds=None, fpreproc=None, as_pandas=True,  
verbose_eval=None, show_stdv=True, seed=0, callbacks=None, shuffle=True)
```

- `params` (dict): 부스터 파라미터
- `dtrain` (DMatrix): 학습 데이터
- `num_boost_round` (int): 부스팅 반복 횟수
- `nfold` (int): CV 폴드 개수.
- `stratified` (bool): CV 수행 시 층화 표본 추출(stratified sampling) 수행 여부
- `metrics` (string or list of strings): CV 수행 시 모니터링할 성능 평가 지표
- `early_stopping_rounds` (int): 조기 종단을 활성화시킴. 반복 횟수 지정.

사이킷런 래퍼 XGBoost의 개요 및 적용

사이킷런 전용 XGBoost 래퍼 클래스

- 사이킷런의 기본 Estimator 그대로 상속 => fit()과 predict()만으로 학습&예측 가능
- GridSearchCV, Pipeline 등 사이킷런의 다른 유틸리티 그대로 사용 가능
- 기존의 다른 ML 알고리즘으로 만들어놓은 프로그램의 알고리즘 클래스만 XGBoost 래퍼 클래스로 바꾸면 그대로 사용 가능
- 분류를 위한 래퍼 클래스 XGBClassifier / 회귀를 위한 래퍼 클래스 XGBRegressor

사이킷런 래퍼 XGBoost의 개요 및 적용

* XGBClassifier는 기존 사이킷런에서 일반적으로 사용하는 하이퍼 파라미터와 호환성 유지하기 위해 기존의 xgboost 모듈에서 사용하던 네이티브 하이퍼 파라미터 몇 개를 변경

- eta → learning_rate
- sub_sample → subsample
- lambda → reg_lambda
- alpha → reg_alpha

* xgboost의 n_estimators와 num_boost_round 하이퍼 파라미터는 서로 동일한 파라미터

(두 개 동시 사용 시)

파이썬 래퍼 XGBoost API => n_estimators 파라미터 무시하고 num_boost_round 파라미터 적용
XGBClassifier와 같은 사이킷런 래퍼 XGBoost 클래스 => n_estimators 파라미터 적용

사이킷런 래퍼 XGBoost의 개요 및 적용

XGBClassifier 클래스의 fit(), predict(), predict_proba() 이용해 학습&예측 수행

앞의 파이썬 래퍼 XGBoost와 동일하게 n_estimators(num_rounds 대응)는 400, learning_rate(eta 대응)는 0.1, max_depth=3으로 설정

[11]
✓ 1초

```
# 사이킷런 래퍼 XGBoost 클래스인 XGBClassifier 임포트
from xgboost import XGBClassifier

# Warning 메시지를 없애기 위해 eval_metric 값을 XGBClassifier 생성 인자로 입력
xgb_wrapper=XGBClassifier(n_estimators=400, learning_rate=0.05, max_depth=3, eval_metric='logloss')
xgb_wrapper.fit(X_train, y_train, verbose=True)
w_preds=xgb_wrapper.predict(X_test)
w_pred_proba=xgb_wrapper.predict_proba(X_test)[:,-1]
```

학습 데이터 => 앞 예제와 다르게 검증 데이터로
분할되기 이전인 X_train과 y_train 이용
테스트 데이터 => 그대로 X_test와 y_test 사용

[12]
✓ 0초

```
get_clf_eval(y_test, w_preds, w_pred_proba)
```



오차 행렬
[[34 3]
[1 76]]

앞 예제보다 더 좋은 평가 결과 !

정확도: 0.9649, 정밀도: 0.9620, 재현율: 0.9870, F1: 0.9744, AUC:0.9954

사이킷런 래퍼 XGBoost의 개요 및 적용

사이킷런 래퍼 XGBoost에서 조기 중단 수행

fit()에 조기 중단 관련 파라미터 입력

[13]
✓ 1초

```
from xgboost import XGBClassifier

xgb_wrapper=XGBClassifier(n_estimators=400, learning_rate=0.05, max_depth=3)
evals=[(X_tr, y_tr), (X_val, y_val)]
xgb_wrapper.fit(X_tr, y_tr, early_stopping_rounds=50, eval_metric='logloss', eval_set=evals, verbose=True)
```

```
ws50_preds=xgb_wrapper.predict(X_test)
ws50_pred_proba=xgb_wrapper.predict_proba(X_test)[:,:1]
```

early_stopping_rounds: 평가 지표 향상될 수 있는 반복 횟수 정의
eval_metric: 조기 중단 위한 평가 지표
(파이썬 래퍼와 달리 학습과 검증 의미하는 문자열 넣어주지 않아도 됨)
eval_set: 성능 평가 수행할 데이터

⇒ [0]	validation_0-logloss:0.65016	validation_1-logloss:0.66189
[1]	validation_0-logloss:0.61131	validation_1-logloss:0.61144
[2]	validation_0-logloss:0.57563	validation_1-logloss:0.59204
[3]	validation_0-logloss:0.54310	validation_1-logloss:0.57329
[4]	validation_0-logloss:0.51323	validation_1-logloss:0.55037
[5]	validation_0-logloss:0.48447	validation_1-logloss:0.52929
[6]	validation_0-logloss:0.45796	validation_1-logloss:0.51534
[7]	validation_0-logloss:0.43436	validation_1-logloss:0.49718
[8]	validation_0-logloss:0.41150	validation_1-logloss:0.48154
[9]	validation_0-logloss:0.39027	validation_1-logloss:0.46990
[10]	validation_0-logloss:0.37128	validation_1-logloss:0.45474
[11]	validation_0-logloss:0.35254	validation_1-logloss:0.44229
[12]	validation_0-logloss:0.33528	validation_1-logloss:0.42965
[13]	validation_0-logloss:0.31893	validation_1-logloss:0.40958
[14]	validation_0-logloss:0.30439	validation_1-logloss:0.39050
[15]	validation_0-logloss:0.29000	validation_1-logloss:0.38254
[16]	validation_0-logloss:0.27651	validation_1-logloss:0.37393
[17]	validation_0-logloss:0.26389	
[18]	validation_0-logloss:0.25210	
[19]	validation_0-logloss:0.24123	

n_estimators 400이지만 400번 반복하지 않고 파이썬 래퍼의 조기 중단과 동일하게 176번째 반복에서 학습 마무리
126번째 반복에서 검증 데이터 세트의 성능 평가인 validation_1-logloss가 가장 낮았고 이후 50번 반복까지 더 이상 성능 향상되지 않아 학습 조기 종료

사이킷런 래퍼 XGBoost의 개요 및 적용

[14]
✓ 0초

```
get_clf_eval(y_test, ws50_preds, ws50_pred_proba)
```



오차 행렬
[[34 3]
[2 75]]

정확도: 0.9561, 정밀도: 0.9615, 재현율: 0.9740, F1: 0.9677, AUC: 0.9933

조기 중단으로 학습된 XGBClassifier의 예측 성능 결과는 파이썬 래퍼의 조기 중단 성능과 동일

사이킷런 래퍼 XGBoost의 개요 및 적용

조기 중단값 너무 급격하게 줄이면 예측 성능 저하될 우려 큼
early_stopping_rounds를 10으로 하면 아직 성능이 향상될 여지가 있음에도 불구하고
10번 반복하는 동안 성능 평가지표 향상되지 않을 시 반복 멈춰 버림 => 충분한 학습
되지 않아 예측 성능 나빠질 수 있음

```
[15]
✓ 1초
# early_stopping_rounds를 10으로 설정하고 재학습
xgb_wrapper.fit(X_tr,y_tr,early_stopping_rounds=10,eval_metric='logloss',eval_set=evals,verbose=True)

ws10_preds=xgb_wrapper.predict(X_test)
ws10_pred_proba=xgb_wrapper.predict_proba(X_test)[:,-1]
get_clf_eval(y_test,ws10_preds,ws10_pred_proba)
```



[0]	validation_0-logloss:0.65016	validation_1-logloss:0.66183
[1]	validation_0-logloss:0.61131	validation_1-logloss:0.63609
[2]	validation_0-logloss:0.57563	validation_1-logloss:0.61144
[3]	validation_0-logloss:0.54310	validation_1-logloss:0.59204
[4]	validation_0-logloss:0.51323	validation_1-logloss:0.57329
[5]	validation_0-logloss:0.48447	validation_1-logloss:0.55037
[6]	validation_0-logloss:0.45796	validation_1-logloss:0.52929
[7]	validation_0-logloss:0.43436	validation_1-logloss:0.51534
[8]	validation_0-logloss:0.41150	validation_1-logloss:0.49718
[9]	validation_0-logloss:0.39027	validation_1-logloss:0.48154
[10]	validation_0-logloss:0.37128	validation_1-logloss:0.46990
[11]	validation_0-logloss:0.35254	validation_1-logloss:0.45474
[12]	validation_0-logloss:0.33528	validation_1-logloss:0.44229
[13]	validation_0-logloss:0.31893	validation_1-logloss:0.42961
[14]	validation_0-logloss:0.30439	validation_1-logloss:0.42065
[15]	validation_0-logloss:0.29000	validation_1-logloss:0.40958
[16]	validation_0-logloss:0.27651	validation_1-logloss:0.39887
[17]	validation_0-logloss:0.26389	validation_1-logloss:0.39050
[18]	validation_0-logloss:0.25210	validation_1-logloss:0.38254
[19]	validation_0-logloss:0.24123	validation_1-logloss:0.37393
[20]	validation_0-logloss:0.23076	validation_1-logloss:0.36789
[21]	validation_0-logloss:0.22091	validation_1-logloss:0.36017

103번째 반복까지만 수행된 후 학습 종료
103번째 반복의 logloss가 0.25991,
93번째 반복의 logloss가 0.25865로,
10번 반복하는 동안 성능 평가 지수 향상 X
더 이상 수행하지 않고 학습 종료
정확도는 early_stopping_rounds=50일 때보다 낮음

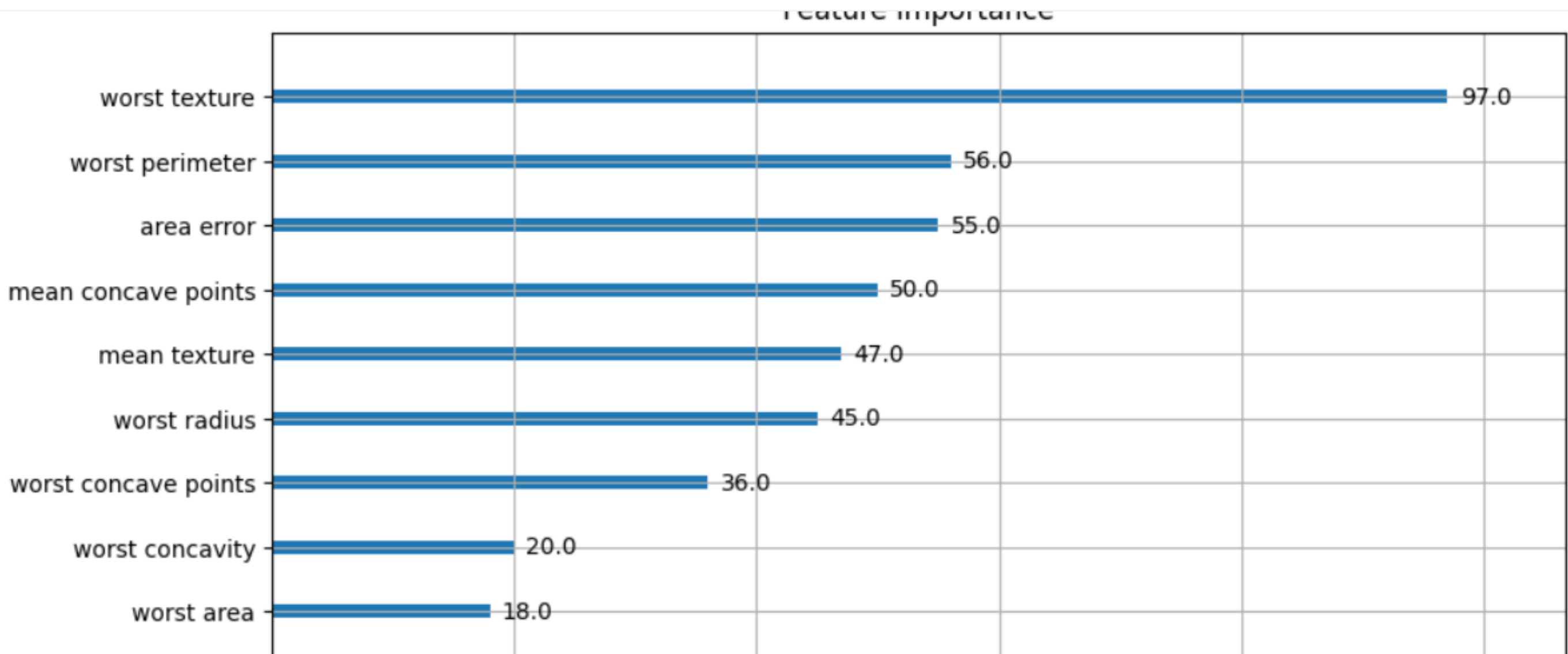
사이킷런 래퍼 XGBoost의 개요 및 적용

[16]
✓ 0초

```
from xgboost import plot_importance
import matplotlib.pyplot as plt
%matplotlib inline

fig,ax=plt.subplots(figsize=(10,12))
# 사이킷런 래퍼 클래스를 입력해도 무방
plot_importance(xgb_wrapper,ax=ax)
```

plot_importance() API에 사이킷런 래퍼 클래스 입력해도
파이썬 래퍼 클래스 입력한 결과와 똑같이 피쳐 중요도 시각화



7. LightGBM



4.7 LightGBM

GBM: 반복 수행을 통해 오류를 최소화할 수 있도록 가중치의 업데이트 값을 도출하는 기법
대표적인 Gradient Boosting 기반 ML 패키지 (1) XGBoost (2) LightGBM

LightGBM (Light Gradient-Boosting Machine)

XGBoost의 장점은 계승, 단점은 보완하는 방식으로 개발

- 더 빠른 학습과 예측 수행 시간
- 더 적은 메모리 사용량
- 대용량 데이터에 대한 뛰어난 예측 성능과 병렬 컴퓨팅 기능 + GPU 지원

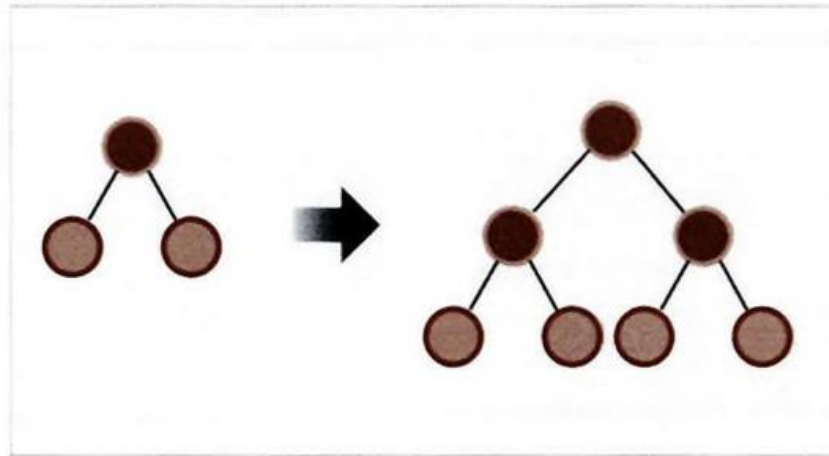
*단점: 적은 데이터 세트에 적용할 경우 과적합이 발생하기 쉬움 (약 10,000건 이하)

파이썬 패키지명: 'lightgbm'

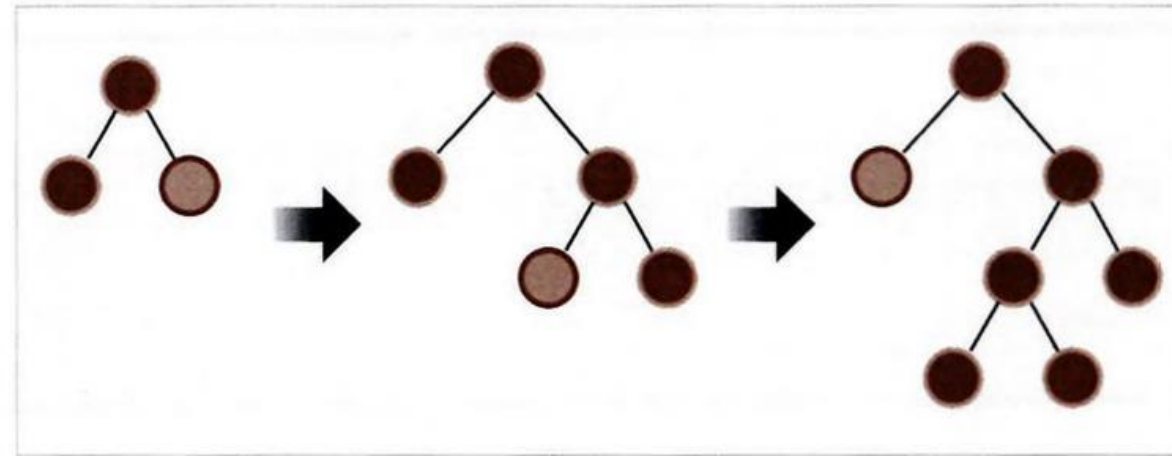
사이킷런 래퍼 LightGBM 클래스: LGBMClassifier & LGBMRegressor

4.7 LightGBM

균형 트리 분할(Level Wise)



리프 중심 트리 분할(Leaf Wise)



LightGBM의 특징: **리프 중심 트리 분할** (leaf wise) 방식 사용

기존 트리 기반 알고리즘: 트리 깊이를 최소화하기 위한 ‘균형 트리 분할 방식’ 사용 → 과적합 방지

***리프 중심 트리 분할:** 트리의 균형을 맞추지 않고, ‘최대 손실 값’ 을 갖는 리프 노드를 계속 분할

트리 깊이가 깊어지고 비대칭적인 규칙 트리가 생성되지만, 학습을 반복할수록 예측 오류 손실 최소화

4.7.2 LightGBM 하이퍼 파라미터

*리프 노드가 계속 분할되며 트리 깊이가 깊어진다는 특성에 맞게 하이퍼 파라미터를 설정해야 함에 유의
*파란색 표시: 기본 튜닝 방안

num_iterations [default: 100]

- 반복 수행하려는 트리 개수 지정
- 크게 지정할수록 예측 성능 높아지나 너무 크면 과적합 사이킷런 호환 클래스에서는 n_estimators

learning_rate [default: 0.1]

- 트리마다 업데이트를 얼마나 강하게 반영할지 조절하는 계수
- 이 값이 작으면 보수적으로 조금씩 업데이트: 각 트리의 영향이 줄어들지만 대신 트리의 개수가 많아야 함
- 베스트 조합: n_estimators ↑ learning_rate ↓

max_depth [default: -1]

- LightGBM은 리프 중심으로 분할하므로 상대적으로 더 깊음
- 0보다 작은 값이면 깊이 제한 X

min_data_in_leaf [default: 20]

- 리프 노드가 되기 위해 필요한 최소한의 레코드 수 (과적합 방지)
- 사이킷런 클래스: min_child_samples

num_leaves [default: 31]

- 하나의 트리가 가질 수 있는 최대 리프 개수

boosting [default: gbdt]

- 부스팅의 트리를 생성하는 알고리즘 기술
- gbdt: gradient boosting decision tree / rf: random forest

bagging_fraction [default: 1.0]

- 데이터 샘플링 비율 지정 (과적합 제어)
- 사이킷런 래퍼 모듈에서는 subsample로 이름 변경

4.7.2 LightGBM 하이퍼 파라미터

feature_fraction [default: 1.0]

- 개별 트리 학습 시 무작위로 선택하는 피처의 비율 (과적합 제어)
- 사이킷런에서는 `colsample_bytree`로 사용됨 (GBM의 `max_features`와 유사: 최적의 분할을 위해 고려할 최대 피처 개수)

lambda_l2 [default: 0.0]

- L2 regulation 제어를 위한 값 (과적합 방지용 패널티)
- 피처의 개수가 많을 경우 적용 검토하며 값이 클수록 과적합 감소 효과 있음 (`reg_lambda`로 변경되어 사용됨)

lambda_l1 [default: 0.0]

- L1 regulation 제어를 위한 값, 과적합 방지 (`reg_alpha`로 사용됨)

objective

- Learning task 파라미터
- 최솟값을 가져야 할 손실함수 정의

4.7.3 하이퍼 파라미터 튜닝 방안

목표: 모델의 복잡도 감소 / 과적합 제어

1. `num_leaves` / `min_data_in_leaf` / `max_depth` 조정

- 개별 트리가 가지는 최대 리프 개수 (증가할수록 트리가 깊어지고 모델 복잡도가 커져 과적합 영향도 커짐)
- 리프 노드 되기 위해 필요한 최소한의 레코드 수, 과적합 제어 (`min_child_samples`)
- 명시적으로 깊이의 크기 제한

2. `n_estimators` ↑ `learning_rate` ↓

- `n_estimators`를 너무 키우면 과적합으로 오히려 성능 저하됨

3. `reg_lambda`, `reg_alpha` 같은 정규화 적용 (`lambda_l1` & `l2`)

4. `colsample_bytree`, `subsample` 적용해 학습 데이터에 쓰일 피처 개수 / 데이터 샘플링 레코드 개수 줄이기

4.7.4 LightGBM(파이썬, 사이킷런), XGBoost 하이퍼 파라미터 비교

사이킷런 호환을 위해 LGBMClassifier & LGBMRegressor 클래스를 래퍼 클래스로 생성
사이킷런 하이퍼 파라미터 명명 규칙에 따라 파라미터명 변경
먼저 생성된 XGBoost에 맞추어 변경하여 동일한 하이퍼 파라미터가 많음

유형	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM	사이킷런 래퍼 XGBoost
파라미터명	num_iterations	n_estimators	n_estimators
	learning_rate	learning_rate	learning_rate
	max_depth	max_depth	max_depth
	min_data_in_leaf	min_child_samples	N/A
	bagging_fraction	subsample	subsample
	feature_fraction	colsample_bytree	colsample_bytree
	lambda_l2	reg_lambda	reg_lambda
	lambda_l1	reg_alpha	reg_alpha
	early_stopping_round	early_stopping_rounds	early_stopping_rounds
	num_leaves	num_leaves	N/A
	min_sum_hessian_in_leaf	min_child_weight	min_child_weight

#4.7.5 LightGBM 적용: 위스콘신 유방암 예측

위스콘신 유방암 예측 실습 - 데이터셋 불러와서 데이터프레임으로 변환

from lightgbm import LGBMClassifier ← LightGBM 분류 클래스 불러오기

import pandas as pd

import numpy as np

from sklearn.datasets import load_breast_cancer # 유방암 진단 데이터 세트

from sklearn.model_selection import train_test_split

dataset = load_breast_cancer()

cancer_df = pd.DataFrame(data=dataset.data, columns=dataset.feature_names) # 환자 샘플의 측정값, 변수명을 Dataframe 형태로 저장

cancer_df['target']=dataset.target # target(=label) 컬럼을 데이터프레임에 추가

X_features = cancer_df.iloc[:, :-1]

y_label = cancer_df.iloc[:, -1]

전체 데이터 중 80%는 학습용, 20%는 테스트용 데이터 추출

X_train, X_test, y_train, y_test = train_test_split(X_features, y_label, test_size=0.2, random_state=156)

위에서 만든 X, y train을 다시 쪼개서 90%는 학습, 10%는 검증용 데이터로 분리

X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.1, random_state=156)

n_estimator 400 설정 (반복 수행하려는 트리 개수 지정 (num_iterations))

lgbm_wrapper = LGBMClassifier(n_estimators=400, learning_rate=0.05)

#4.7.5 LightGBM 적용: 위스콘신 유방암 예측

학습용 + 검증용 데이터 지정과 학습, 조기 중단 수행

```
evals = [(X_tr, y_tr), (X_val, y_val)]
```

```
lgbm_wrapper.fit(X_tr, y_tr, early_stopping_rounds=50, eval_metric="logloss", eval_set=evals, verbose=True)
```

- evals에 학습용, 검증용 데이터 지정 (성능 체크 용도)
- X_val, y_val에서 50번 동안 성능 개선이 없으면 학습을 멈춤 → 과적합 방지
- 평가 지표로 logloss(로그 손실) 사용, 학습 진행 상황을 단계별로 출력

예측

```
preds = lgbm_wrapper.predict(X_test)
```

```
pred_proba = lgbm_wrapper.predict_proba(X_test)[:, 1]
```

- preds: 학습이 끝난 모델을 사용해 테스트 데이터(X_test)에 대해 예측한 결과
- predict_proba(): 각 샘플이 각 클래스에 속할 확률을 반환
- pred_proba: 1(양성) 클래스일 확률만 가져와 저장한 값

```
[101] training's binary_logloss: 0.0122421    valid_1's binary_logloss: 0.276695
[102] training's binary_logloss: 0.0118067    valid_1's binary_logloss: 0.278488
[103] training's binary_logloss: 0.0113994    valid_1's binary_logloss: 0.278932
[104] training's binary_logloss: 0.0109799    valid_1's binary_logloss: 0.280997
[105] training's binary_logloss: 0.0105953    valid_1's binary_logloss: 0.281454
[106] training's binary_logloss: 0.0102381    valid_1's binary_logloss: 0.282058
[107] training's binary_logloss: 0.00986714   valid_1's binary_logloss: 0.279275
[108] training's binary_logloss: 0.00950998   valid_1's binary_logloss: 0.281427
[109] training's binary_logloss: 0.00915965   valid_1's binary_logloss: 0.280752
[110] training's binary_logloss: 0.00882581   valid_1's binary_logloss: 0.282152
[111] training's binary_logloss: 0.00850714   valid_1's binary_logloss: 0.280894
```

<출력 결과>

조기 중단으로 111번 반복까지만 수행하고 학습 종료

평가 지표로 logloss 사용, verbose=True 설정으로 진행 단계별로 출력

#4.7.5 LightGBM 적용: 위스콘신 유방암 예측

```
# 학습된 LGBM 모델 기반으로 예측 성능 평가
get_clf_eval(y_test, preds, pred_proba)
```

오차 행렬

```
[[34  3]
 [ 2 75]]
```

정확도: 0.9561, 정밀도: 0.9615, 재현율: 0.9740, F1: 0.9677, AUC:0.9877

- get_clf_eval(): 분류 성능 평가 함수
- pred(0/1 예측)로 계산 가능한 지표: Accuracy, Precision, Recall, F1
- proba(확률)로 계산 가능한 지표: ROC-AUC

피처 중요도 시각화하는 내장 API: `plot_importance`

```
from lightgbm import plot_importance
```

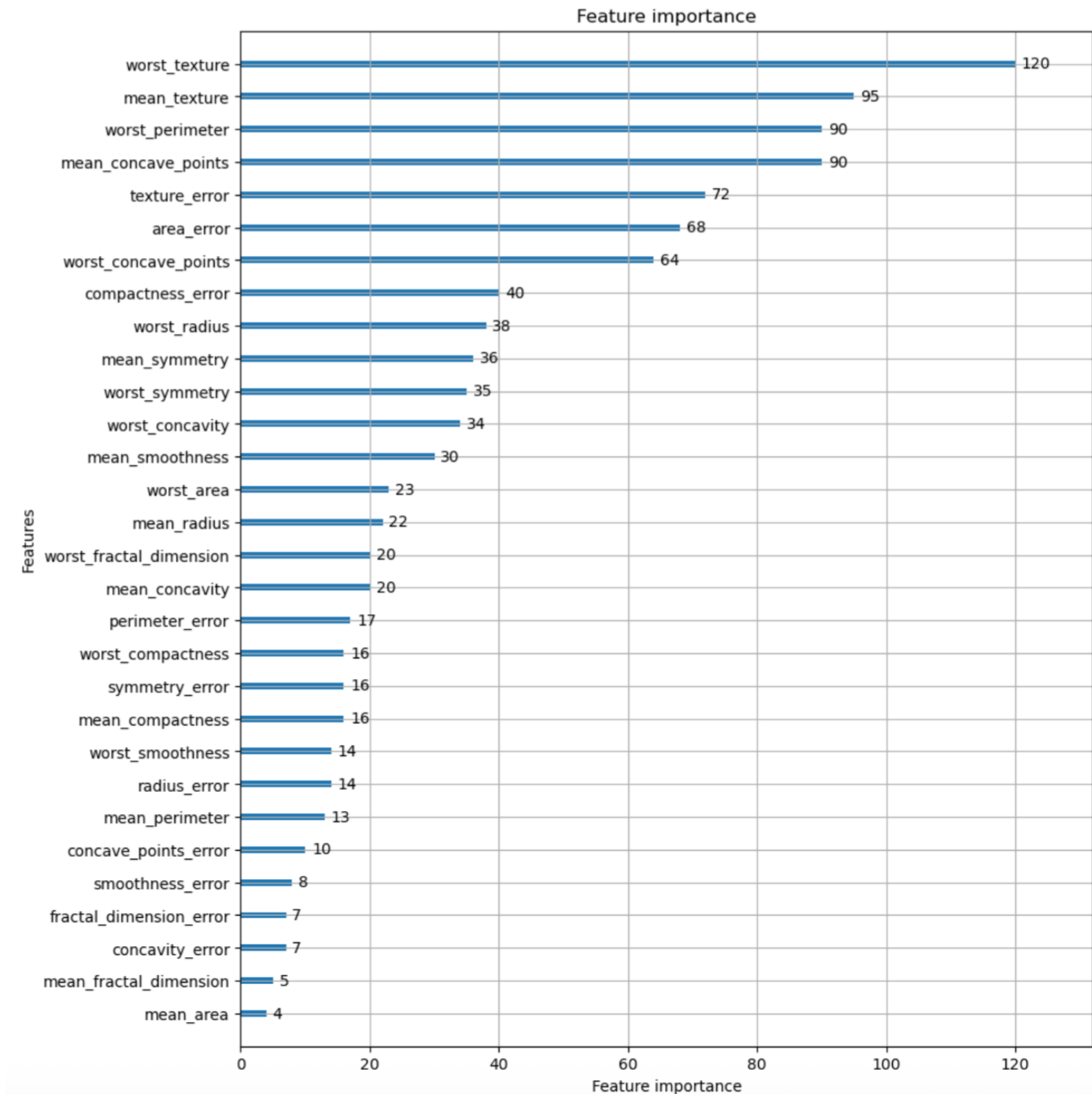
```
import matplotlib.pyplot as plt
```

```
%matplotlib inline
```

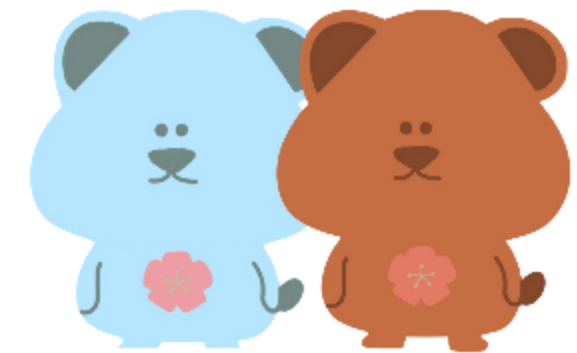
```
fig, ax = plt.subplots(figsize=(10, 12)) # 그림 크기 설정
```

```
plot_importance(lgbm_wrapper, ax=ax)
```

- 막대에 표시되는 기준: split (각 피처가 트리에서 얼마나 자주 사용되었는지)
- 막대가 길수록 → 해당 특징이 모델 예측에 더 크게 기여



8. 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝



#4.8.1 베이지안 최적화 개요

기존의 하이퍼 파라미터 튜닝에 사용되던 방식: 사이킷런의 Grid Search 방식

* Grid Search: 모델에 적용할 하이퍼 파라미터 후보 값들을 딕셔너리 형태의 그리드로 정의하고, fit() 메서드 호출 시에 정의된 하이퍼 파라미터 조합의 모든 경우에 대해 교차 검증을 수행하며 모델 학습, 교차 검증 결과 최고 성능을 보인 하이퍼 파라미터 조합을 선택하는 방식 (최적화 시간이 길다는 단점)

XGBoost / LightGBM: 다른 알고리즘에 비해 많은 하이퍼 파라미터 개수 → 최적 하이퍼 파라미터 찾으려면 많은 시간이 소모됨

```
params = {  
    'max_depth' = [10, 20, 30, 40, 50], 'num_leaves' = [35, 45, 55, 65],  
    'colsample_bytree' = [0.5, 0.6, 0.7, 0.8, 0.9], 'subsample' = [0.5, 0.6, 0.7, 0.8, 0.9],  
    'min_child_weight' = [10, 20, 30, 40], reg_alpha = [0.01, 0.05, 0.1]  
}
```

5개 * 4개 * 5개 * 5개 * 4개 * 3개
Grid Search 적용 시 6000회에 걸쳐 학습·평가 반복
수행 시간 매우 오래 걸림

실무의 대용량 학습 데이터에 하이퍼 파라미터 튜닝 시 Grid Search가 아닌 다른 방식을 도입
“베이지안 최적화 기법”

#4.8.1 베이지안 최적화 개요

베이지안 최적화

미지의 함수가 반환하는 값의 최소/최댓값을 만드는 최적의 입력값을 최대한 적은 시도를 통해 빠르게 찾는 최적화 방식

$$f(x, y) = 2x - 3y$$

함수의 반환 값을 최대/최소로 하는 x, y 값을 찾아라
($0 \leq x, y \leq 100$)

x 는 100, y 는 0일 때 최댓값 / x 는 0, y 는 100일 때 최솟값 반환
만약 함수 식 자체를 알 수 없고, 입력값과 반환값만 알 수 있다면?
입력값의 개수가 많거나 범위가 넓다면?

이러한 경우에 베이지안 최적화를 사용하면 쉽고 빠르게 최적 입력값을 찾을 수 있음

베이지안 최적화는 **베이지안 확률**에 기반을 둔 최적화 기법

#4.8.1 베이지안 최적화 개요

베이지안 확률

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

‘A라는 사건이 발생했을 때, B 사건이 발생할 확률’: 조건부 확률에서 출발

두 확률 변수의 사전 확률과 사후 확률 사이의 관계를 나타내는 베이즈 정리를 이용한 통계학 접근

일어나지 않은 일에 대한 확률을 사건과 유관한 여러 가지 확률을 이용해 추정하는 것

〈새로운 사건의 관측이나 새로운 샘플 데이터를 기반으로 사후 확률을 개선해 나가는 방식〉

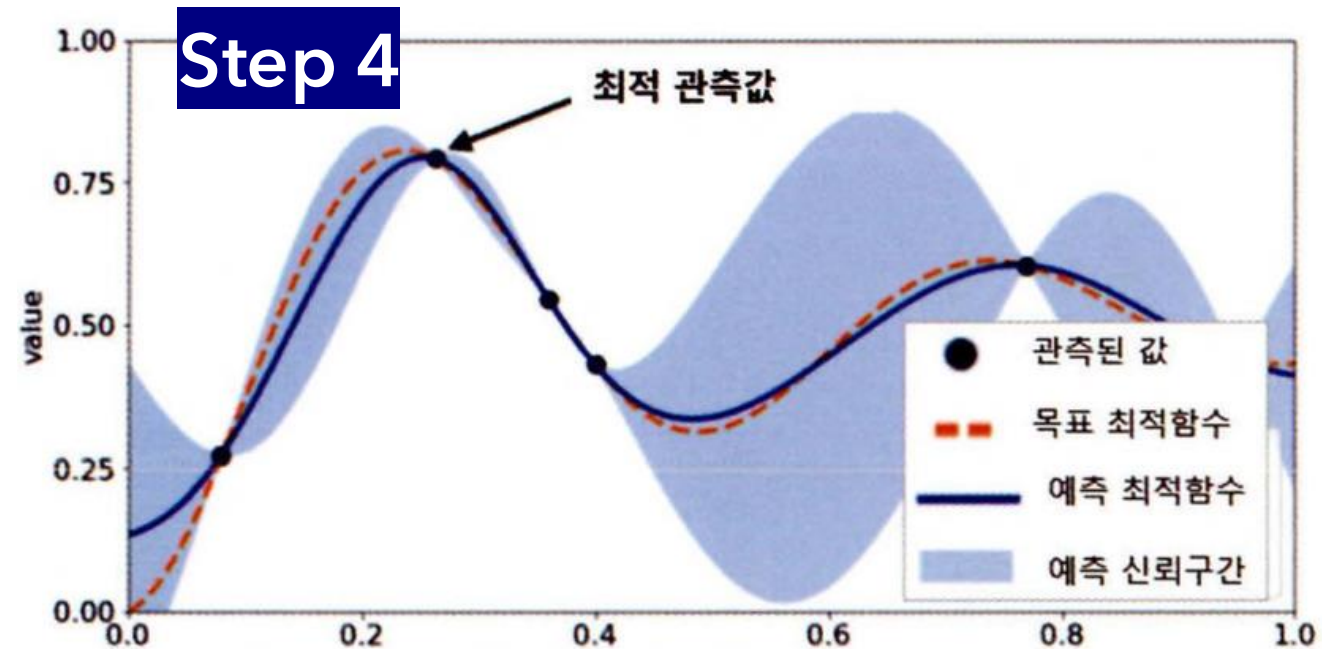
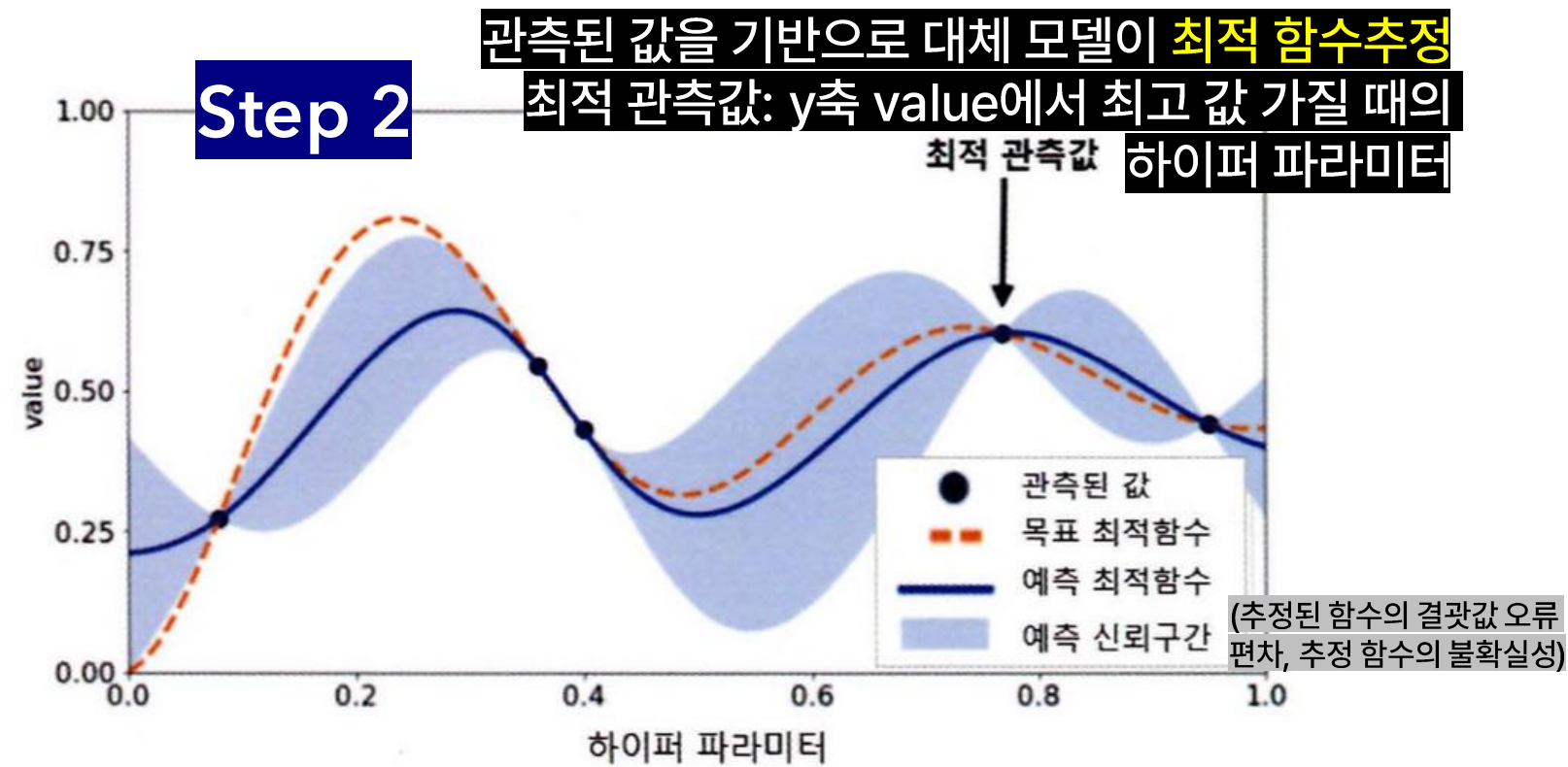
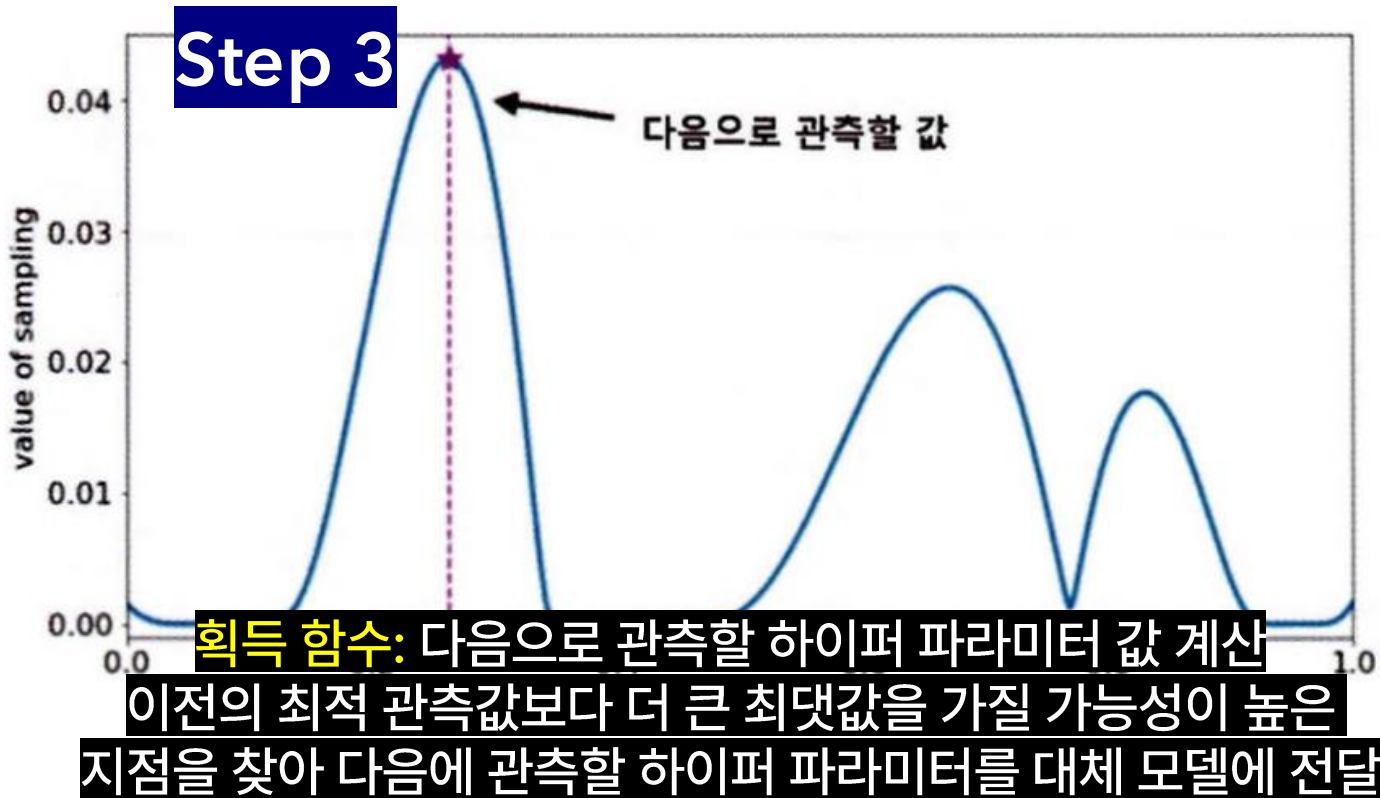
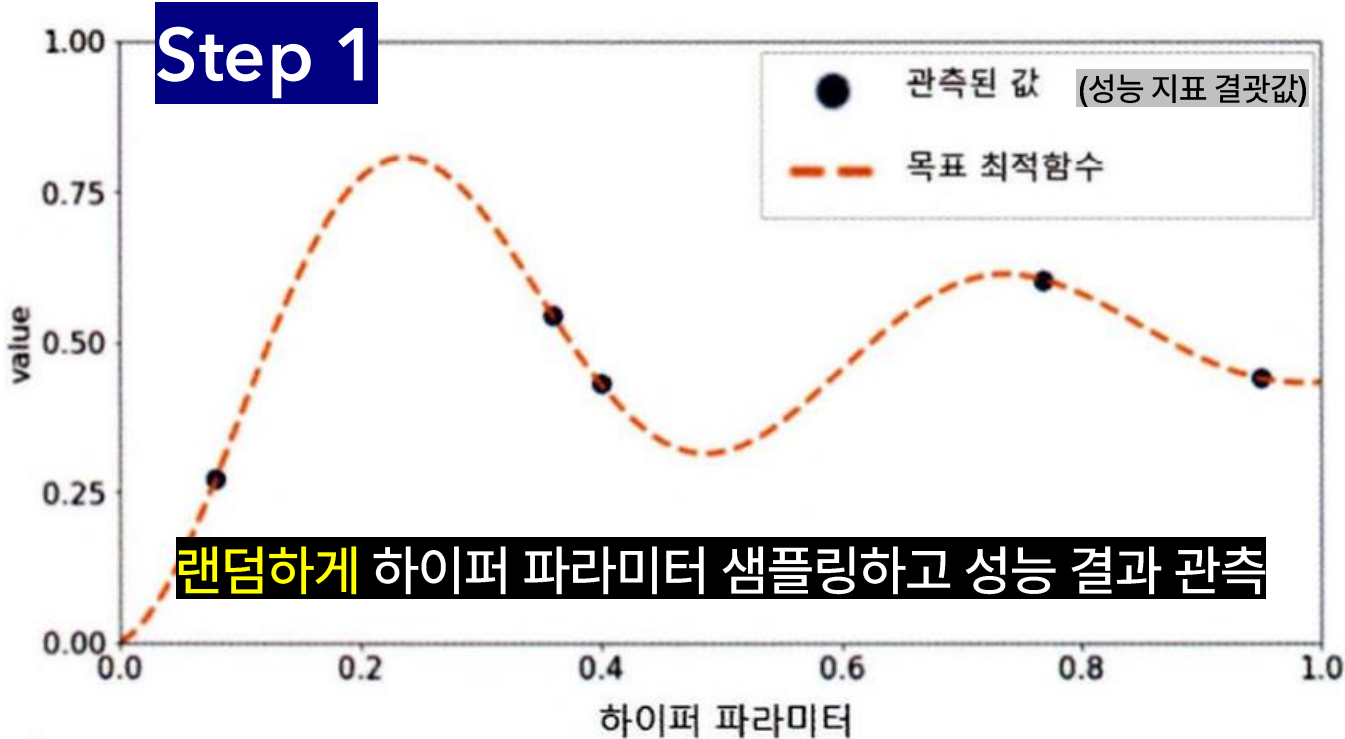
베이지안 최적화

새로운 데이터를 입력받았을 때 최적 함수를 예측하는 사후 모델을 개선해 나가며 최적 함수 모델을 생성

[구성 요소]

- ① 대체 모델 (Surrogate Model) : 획득 함수로부터 최적 함수를 예측할 수 있는 입력값을 추천 받은 뒤 이를 기반으로 최적 함수 모델을 개선
- ② 획득 함수(Acquisition Function) : 개선된 대체 모델을 기반으로 최적 입력값 계산

#4.8.1 베이지안 최적화 개요: 단계



획득 함수로부터 전달된 하이퍼 파라미터를 수행해 관측된 값을 기반으로
대체 모델 갱신, 다시 최적 함수를 예측 추정

#4.8.2 HyperOpt 사용하기

HyperOpt

ML 모델의 하이퍼 파라미터 튜닝에 베이지안 최적화를 적용할 수 있게 제공되는 대표적인 파이썬 패키지

[주요 로직]

1. 입력 변수명, 입력값의 검색 공간 설정
2. 목적 함수 설정
3. 목적 함수의 **최솟값**을 가지는 최적 입력값 유추
(다른 패키지: 최댓값을 가지는 입력값 유추)

1. 검색 공간 설정

HyperOpt의 hp 모듈 이용

- 입력 변수명, 입력값 검색 공간은 파이썬 딕셔너리 형태로
- Key: 입력 변수명
- Value: 해당 입력 변수의 검색 공간
- hp 모듈: 검색 공간을 다양하게 설정할 수 있도록 여러 함수 제공

`hp.quniform(label, low, high, q)`

label로 지정된 입력값 변수 공간을 low에서 high까지 q 간격을 가지고 설정

`hp.uniform(label, low, high)`

low~high까지 정규 분포 형태의 검색 공간 설정

`hp.radint(label, upper)`

0~upper까지 랜덤한 정숫값으로 검색 공간 설정

#4.8.2 HyperOpt 사용하기

2. 목적 함수 생성

변수 값과 검색 공간을 가지는 딕셔너리를 인자로 받고, 특정 값을 반환하는 구조로 만들어져야 함

```
from hyperopt import hp
from hyperopt import STATUS_OK

# 입력값 검색 공간 지정: -10~10까지 1 간격을 가지는 입력 변수 x , -15~15까지 1 간격 가지는 입력 변수 y 설정
search_space = {'x': hp.quniform('x', -10, 10, 1), 'y': hp.quniform('y', -15, 15, 1)}

# 목적 함수 생성: search_space로 지정된 딕셔너리에서 x, y 입력 변수값을 추출해 retval로 계산된 값을 반환함
def objective_func(search_space):
    x = search_space['x']
    y = search_space['y']
    retval = x**2 - 20*y

    return retval
```


#4.8.2 HyperOpt 사용하기

3. 목적 함수의 최솟값을 가지는 최적 입력값 유추

앞서 설정한 목적 함수의 반환값이 최소가 되는 최적의 입력값을 베이지안 최적화 기법에 기반해 찾아야 함
HyperOpt에서 제공하는 `fmin(objective, space, algo, max_evals, trials)` 함수 활용

`fmin()` 함수의 주요 인자:

1. `fn`: 목적 함수 (앞 슬라이드에서 생성한 `objective_func`와 같음)
2. `space`: 검색 공간 딕셔너리 (= `search_space`)
3. `algo`: 베이지안 최적화 적용 알고리즘. 기본적으로 `tpe.suggest` 사용 – HyperOpt의 기본 최적화 알고리즘(Tree of Parzen Estimator)
4. `max_evals`: 최적 입력값을 찾기 위한 입력값 시도 횟수
5. `trials`: 최적 입력값을 찾기 위해 시도한 입력값 및 해당 입력값의 목적 함수 반환값 결과를 저장하는 데 사용. Trials 클래스를 객체로 생성한 변수명을 입력함.
6. `rstate`: `fmin()` 수행 시마다 동일한 결괏값 가지도록 설정하는 랜덤 seed 값

#4.8.2 HyperOpt 사용하기

3. 목적 함수의 최솟값을 가지는 최적 입력값 유추

```
from hyperopt import fmin, tpe, Trials
```

```
# Trials 객체값 생성: 최적 입력값을 찾기 위해 시도한 입력값 및 해당 입력값의  
목적 함수 반환값 결과를 저장
```

```
trial_val = Trials()
```

```
# 목적 함수의 최솟값을 반환하는 최적 입력 변수값을 5번의 입력값 시도  
(max_evals=5)로 찾아냄
```

```
best_01 = fmin(fn=objective_func, space=search_space,  
algo=tpe.suggest, max_evals=5,  
              trials=trial_val, rstate=np.random.default_rng(seed=0))  
print('best', best_01)
```

```
100%|████████████████████████████████████████████████████████████████████████████████| 5/5 [00:00<00:00, 476.91trial/s, best loss: -224.0]  
best {'x': np.float64(-4.0), 'y': np.float64(12.0)}
```

목적 함수가 반환하는 값: $x^2 - 20y$

x는 0, y는 15에 가까울수록 반환값이 최소로 근사될 것

best_01의 변수값: x는 -4.0, y는 12.0

최적의 값은 아니지만 5번의 수행으로 어느 정도 최적값에 가까워짐

```
# max_evals 20회로 늘려 다시 테스트
```

```
best_02 = fmin(fn=objective_func, space=search_space,  
algo=tpe.suggest, max_evals=20,  
              trials=trial_val, rstate=np.random.default_rng(seed=0))  
print('best', best_02)
```

```
100%|████████████████████████████████████████████████████████████████████████████████| 20/20 [00:00<00:00, 1107.49trial/s, best loss: -296.0]  
best {'x': np.float64(2.0), 'y': np.float64(15.0)}
```

max_evals 20회로 늘리자 (*최적 입력값을 찾기 위한 입력값 시도 횟수)

best_02의 변수값: x는 2.0, y는 15.0

목적 함수의 최적 최솟값을 근사할 수 있는 결과가 도출됨

만약 Grid Search 방식이었다면... $21 \times 31 = 651$ 회 반복 필요했을 것

#4.8.2 HyperOpt 사용하기

3. 목적 함수의 최솟값을 가지는 최적 입력값 유추

Trials 객체: 함수 반복 수행 시마다 입력되는 변수값 & 함수 반환값을 속성으로 가짐

중요 속성: results, vals – fmin() 함수 수행 시마다 최적화되는 경과를 볼 수 있는 함수 반환값 & 입력 변수값의 정보 제공

- results: 함수 반복 수행 시마다 반환되는 반환값
- vals: 함수 반복 수행 시마다 입력되는 입력 변수값

```
print(trial_val.results)
```

```
[{'loss': -64.0, 'status': 'ok'}, {'loss': -184.0, 'status': 'ok'}, {'loss': 56.0, 'status': 'ok'}, {'loss': 61.0, 'status': 'ok'}, {'loss': -64.0, 'status': 'ok'}, {'loss': 56.0, 'status': 'ok'}, {'loss': -224.0, 'status': 'ok'}, {'loss': 61.0, 'status': 'ok'}, {'loss': -40.0, 'status': 'ok'}, {'loss': 281.0, 'status': 'ok'}, {'loss': 100.0, 'status': 'ok'}, {'loss': 60.0, 'status': 'ok'}, {'loss': -39.0, 'status': 'ok'}, {'loss': -164.0, 'status': 'ok'}, {'loss': 21.0, 'status': 'ok'}]
```

```
print(trial_val.vals)
```

```
{'x': [np.float64(-6.0), np.float64(-4.0), np.float64(4.0), np.float64(-4.0), np.float64(9.0), np.float64(-4.0), np.float64(4.0), np.float64(-4.0), np.float64(9.0), np.float64(2.0), np.float64(0.0), np.float64(-8.0), np.float64(-0.0), np.float64(-0.0), np.float64(1.0), np.float64(9.0), np.float64(9.0)], 'y': [np.float64(5.0), np.float64(10.0), np.float64(-2.0), np.float64(12.0), np.float64(0.0), np.float64(10.0), np.float64(-2.0), np.float64(12.0), np.float64(1.0), np.float64(15.0), np.float64(-10.0), np.float64(0.0), np.float64(-5.0), np.float64(-3.0), np.float64(2.0), np.float64(3.0)]}
```

[results 속성]

리스트 내부의 개별 원소:

{'loss': 함수 반환값, 'status': 반환 상태값}

max_evals=20이었으므로 results 속성은

20개의 딕셔너리를 개별 원소로 갖는 리스트로 구성되어 있음을 알 수 있음.

[vals 속성]

딕셔너리 형태

- key: 입력 변수 x, y
- value: 20회의 반복 수행 시마다 사용되는 입력값들을 리스트 형태로 가짐

#4.8.2 HyperOpt 사용하기

*results, vals 속성값들을 DataFrame으로 만들어 직관적으로 수행 횟수별 입력 변수 x, y와 반환값 loss 값 확인하기

```
import pandas as pd

# results의 loss 키값에 해당하는 value들을 추출해 리스트로 생성
losses = [loss_dict['loss'] for loss_dict in trial_val.results]

# DF로 생성
result_df = pd.DataFrame({'x': trial_val.vals['x'], 'y': trial_val.vals['y'], 'losses': losses})
result_df
```

[실행 결과]

	x	y	losses
0	-6.0	5.0	-64.0
1	-4.0	10.0	-184.0
2	4.0	-2.0	56.0
3	-4.0	12.0	-224.0
15	-0.0	-3.0	60.0
16	1.0	2.0	-39.0
17	9.0	4.0	1.0
18	6.0	10.0	-164.0
19	9.0	3.0	21.0

[vals]

입력 변수 x값: x 컬럼으로 생성, 컬럼값으로 vals 속성의 x 키값에 해당하는 value 할당
입력 변수 y값: y 컬럼으로 생성, 컬럼값으로 vals 속성의 y 키값에 해당하는 value 할당

[results]

함수 반환 값: loss 컬럼으로 생성, results 속성에서 loss 키값에 해당하는 value 할당

#4.8.3 HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화 과정 3단계

1. 적용해야 할 하이퍼 파라미터와 검색 공간(탐색할 파라미터 범위) 설정
2. 목적 함수를 만들고 XGBoost 학습
3. 예측 성능 결과 반환 값으로 설정
 - fmin() 함수가 이 목적 함수를 반복적으로 실행하면서 탐색 공간 안에서 가장 좋은 파라미터 조합을 찾아냄

*주의할 점

1. 특정 하이퍼 파라미터들은 입력을 정수값으로만 받는데, HyperOpt는 입력/반환값이 모두 실수형이기 때문에 하이퍼 파라미터 입력 시 형변환 필요
2. HyperOpt의 목적 함수는 최솟값을 반환하도록 최적화해야 하기 때문에, 성능값이 클수록 좋은 지표의 경우 -1을 곱해야 함 (예: 정확도 0.8 vs 0.9)

#4.8.3 HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

1. 하이퍼 파라미터 검색 공간 설정

이전 예제의 유방암 데이터 세트 로딩하여 학습, 검증, 테스트 세트로 분리

hp.quniform(label, low, high, q), 정수형 파라미터에 적용
label로 지정된 입력값 변수 공간을 low에서 high까지 q 간격으로 설정

```
from hyperopt import hp
```

hp.uniform(label, low, high)
low~high까지 정규 분포 형태의 검색 공간 설정

```
# max_depth는 5에서 20까지 1 간격, min_child_weight는 1에서 2까지 1 간격
```

```
# colsample_bytree는 0.5에서 1 사이, learning_rate는 0.01에서 0.2 사이 정규 분포된 값을 검색
```

```
xgb_search_space = {'max_depth': hp.quniform('max_depth', 5, 20, 1),  
                    'min_child_weight': hp.quniform('min_child_weight', 1, 2, 1),  
                    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2),  
                    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 1),  
                    }
```

fmin() 함수가
탐색할 공간

#4.8.3 HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

2. 목적 함수 설정

```
from sklearn.model_selection import cross_val_score # 3개의 교차 검증 세트로 정확도 반환
from xgboost import XGBClassifier
from hyperopt import STATUS_OK
```

주의사항 1. 정수형 하이퍼 파라미터값 설정 시 정수형으로 형 변환 필요
(5, 20, 1) 적용해도 검색되는 값은 [5.0, 6.0, 7.0...] 같은 실수 값

```
def objective_func(search_space):
    xgb_clf = XGBClassifier(n_estimators=100, max_depth=int(search_space['max_depth']),
                           min_child_weight=int(search_space['min_child_weight']),
                           learning_rate=search_space['learning_rate'],
                           colsample_bytree=search_space['colsample_bytree'],
                           eval_metric='logloss')

    accuracy = cross_val_score(xgb_clf, X_train, y_train, scoring='accuracy', cv=3)

    return {'loss': -1 * np.mean(accuracy), 'status': STATUS_OK}
```

주의사항 2. 반환값이 accuracy: 클수록 좋은 값. 정확도 0.8 vs 0.9
fmin() 함수는 '최솟값'을 최적화하기 때문에 0.8을 더 좋은 최적화로 판단
-1을 곱하면 -0.8 vs -0.9: 결과 달라짐

#4.8.3 HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

3. fmin() 이용해 최적 하이퍼 파라미터 도출

```
from hyperopt import fmin, tpe, Trials

trial_val = Trials()
best = fmin(fn=objective_func,
            space=xgb_search_space, 앞에서 설정한 검색 공간 안에서 탐색
            algo=tpe.suggest,
            max_evals=50, # 최대 반복 수행 횟수 지정
            trials=trial_val, rstate=np.random.default_rng(seed=9))
print('best:', best)
```

```
100%|████████████████████| 50/50 [00:11<00:00, 4.25trial/s, best loss: -0.9692401533635412]
best: {'colsample_bytree': 0.548301545497125, 'learning_rate': 0.1840281762576621, 'max_depth': 18.0, 'min_child_weight': 2.0}
```

```
print('colsample_bytree:{0}, learning_rate:{1}, max_depth:{2},
min_child_weight:{3}'.format(
# 정수형 하이퍼 파라미터는 정수형으로 변환 + 실수형 하이퍼 파라미터는 소수점 5자리까지 변환
round(best['colsample_bytree'], 5), round(best['learning_rate'], 5),
int(best['max_depth']), int(best['min_child_weight'])))
```

반복 탐색 단계 (50회)

1. fmin의 탐색 알고리즘 tpe.suggest가 xgb_search_space 안에서 하이퍼 파라미터 값 조합을 하나 선택, 이를 인자로 넣어 objective_func 호출
2. 목적 함수 내에서 그 인자로 모델을 학습시키고 교차 검증하여 성능 평가
3. 계산된 성능 점수: loss로 변환되어 fmin에게 반환 - Trials() 객체에 해당 하이퍼 파라미터 조합과 결과를 저장
4. 50번 반복, Trials() 객체에 저장된 결과 중 loss 값이 가장 낮은(=정확도가 높은) 조합을 최종 선택

#4.8.3 HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

4. 성능 평가

```
xgb_wrapper = XGBClassifier(n_estimators=400,  
                             learning_rate=round(best['learning_rate'], 5),  
                             max_depth=int(best['max_depth']),  
                             min_child_weight=int(best['min_child_weight']),  
                             colsample_bytree=round(best['colsample_bytree'], 5)  
                             )
```

앞 단계에서 도출된 최적 하이퍼 파라미터(best)
적용해 XGBClassifier 재학습

```
evals = [(X_tr, y_tr), (X_val, y_val)]  
xgb_wrapper.fit(X_tr, y_tr, early_stopping_rounds=50, eval_metric='logloss',  
                eval_set=evals, verbose=True)
```

```
preds = xgb_wrapper.predict(X_test)  
pred_proba = xgb_wrapper.predict_proba(X_test)[: , 1]
```

```
get_clf_eval(y_test, preds, pred_proba)
```

```
[180] validation_0-logloss:0.01297 validation_1-logloss:0.26005  
[181] validation_0-logloss:0.01295 validation_1-logloss:0.25997  
[182] validation_0-logloss:0.01294 validation_1-logloss:0.26040  
[183] validation_0-logloss:0.01292 validation_1-logloss:0.26016  
[184] validation_0-logloss:0.01290 validation_1-logloss:0.25977  
[185] validation_0-logloss:0.01288 validation_1-logloss:0.25927  
[186] validation_0-logloss:0.01286 validation_1-logloss:0.25951  
[187] validation_0-logloss:0.01284 validation_1-logloss:0.25943  
[188] validation_0-logloss:0.01283 validation_1-logloss:0.25921
```

오차 행렬

```
[[34  3]  
 [ 3 74]]
```

정확도: 0.9474, 정밀도: 0.9610, 재현율: 0.9610, F1: 0.9610, AUC:0.9923

10. 분류 실습



#4.10 분류 실습 – 캐글 신용카드 사기 검출

#1 Kaggle의 신용카드 데이터 세트를 이용해 신용카드 사기 검출 분류 실습을 수행

↳ Class속성(레이블)은 매우 불균형한 분포를 가짐.

0 과 1 으로 분류되는데, 전체 데이터의 약 0.172%만이 레이블 값이 1, 즉 신용카드 사기 트랜잭션

#2 언더 샘플링과 오버 샘플링의 이해

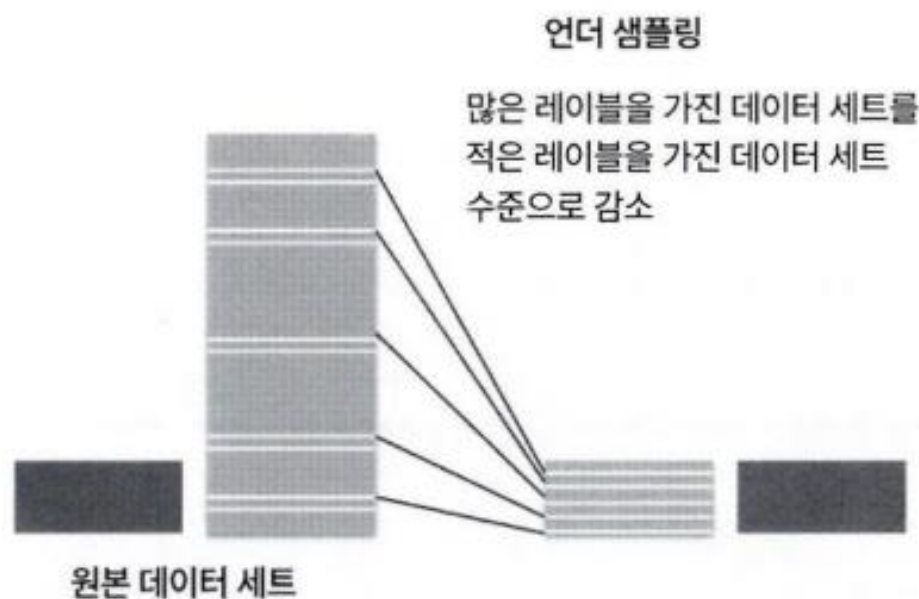
레이블이 불균형한 분포를 가진 데이터 세트를 학습시킬 때 예측 성능의 문제가 발생

이상 레이블을 가지는 데이터 건수는 매우 적기 때문에 제대로 다양한 유형을 학습하지 못하므로
일반적으로 정상 레이블로 치우친 학습을 수행해 제대로 된 이상 데이터 검출이 어려워지기 쉬움

⇒ 적절한 학습 데이터를 확보하는 대표적인 방안 :

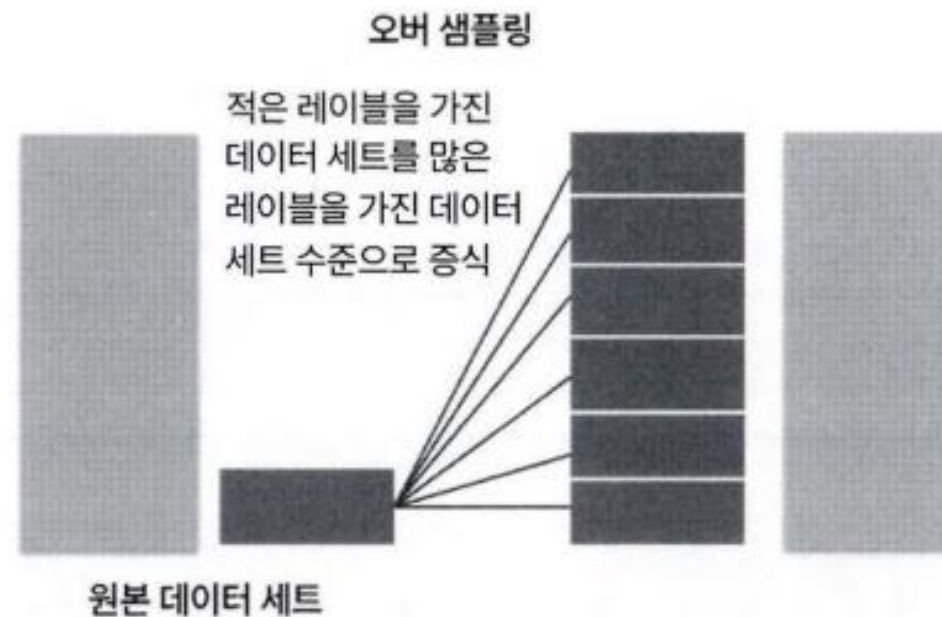
언더샘플링, 오버 샘플링

<언더 샘플링>



- 정상 레이블 데이터를 이상 레이블 데이터 수준으로 줄여 버린 상태에서 학습을 수행하면 과도하게 정상 레이블로 학습/예측하는 부작용을 개선
- 너무 많은 정상 레이블 데이터를 감소시켜서 정상 레이블의 경우 제대로 된 학습을 수행할 수 없는 문제가 발생할 수도 있으므로 유의

<오버 샘플링>



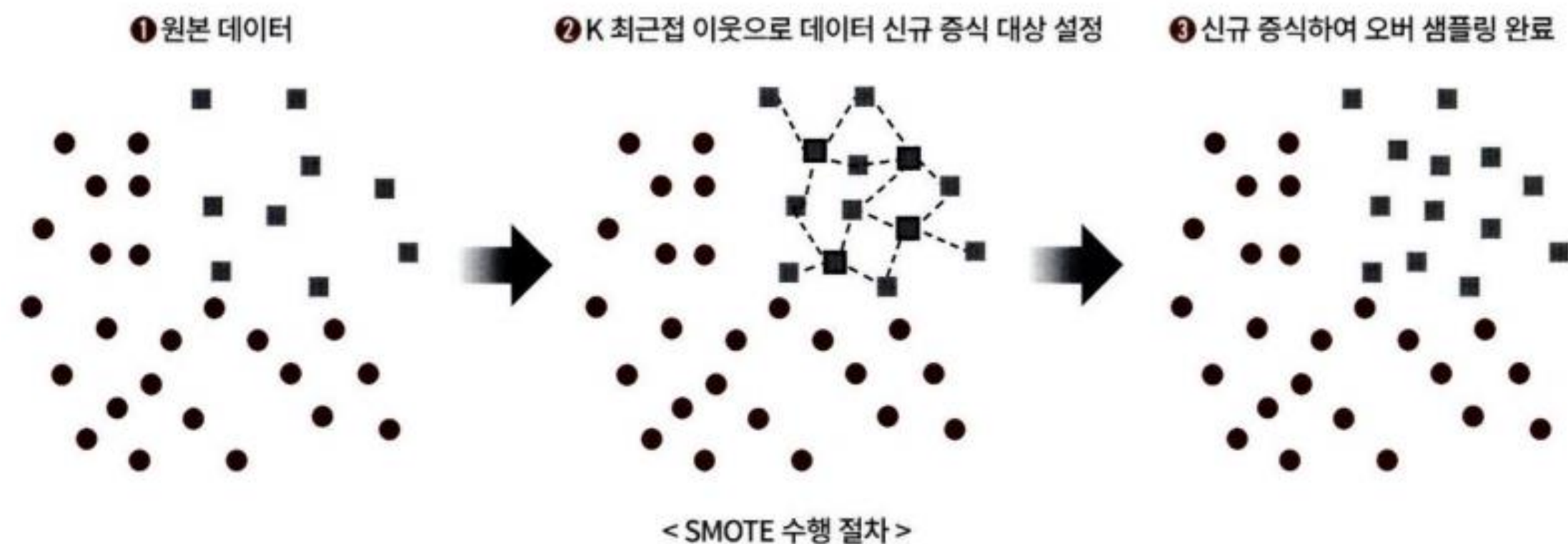
- 동일한 데이터를 단순히 증식하는 방법은 과적합(Overfitting)이 되기 때문에 의미가 없으므로 원본 데이터의 피쳐 값들을 아주 약간만 변경하여 증식
- 대표적으로 **SMOTE(Synthetic Minority Over-sampling Technique)** 방법이 있음

#4.10 분류 실습 – 캐글 신용카드 사기 검출

#3 SMOTE(Synthetic Minority Over-sampling Technique) 방법

: 적은 데이터 세트에 있는 개별 데이터들의 K 최근접 이웃(K Nearest Neighbor)을 찾아서 이 데이터와 K개 이웃들의 차이를 일정 값으로 만들어서 기존 데이터와 약간 차이가 나는 새로운 데이터들을 생성하는 방식

SMOTE를 구현한 대표적인 파이썬 패키지는 imbalanced-learn



1. 소수 클래스 샘플 집합 S 선택.
2. 각 소수 샘플 $x \in S$ 에 대해 k-최근접 이웃(k-NN)을 소수 클래스 내에서 찾음.
3. 얼마나 많은 synthetic sample을 생성할지(목표 수) 계산.
4. 각 x 에 대해 필요한 만큼 반복:
 - k 이웃 중 하나 x_{nbr} 를 무작위 선택.
 - $\delta \in [0, 1)$ 을 무작위 추출.
 - $x_{new} = x + \delta(x_{nbr} - x)$ 생성.
5. 새로 생성한 샘플들을 원래 데이터의 소수 클래스에 추가.

$$x_{new} = x_i + \delta \cdot (x_{i,neighbor} - x_i), \delta \sim \text{Uniform}(0, 1)$$

(예시)

$x_i = (1.0, 1.0)$, 이웃 $x_j = (2.0, 2.0)$

$\delta = 0.3$ 이면 $x_{new} = (1 + 0.3(2 - 1), 1 + 0.3(2 - 1)) = (1.3, 1.3)$.

#4.10 분류 실습 – 캐글 신용카드 사기 검출

#4 데이터 일차 가공 및 모델 학습/예측/평가

<데이터 사전 가공 후 학습/테스트 데이터 세트 생성>

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
%matplotlib inline

card_df = pd.read_csv('./creditcard.csv')
card_df.head(3)
```

	Time	V1	V2	V3	V4	V9	...	V21	V22
0	0.0	-1.359807	-0.072781	2.536347	1.378155	0.363787	...	-0.018307	0.277838
1	0.0	1.191857	0.266151	0.166480	0.448154	-0.255425	...	-0.225775	-0.638672
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-1.514654	...	0.247998	0.771679

3 rows × 31 columns

```
from sklearn.model_selection import train_test_split
```

```
# 인자로 입력받은 DataFrame을 복사한 뒤 Time 칼럼만 삭제하고 복사된 DataFrame 반환
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    df_copy.drop('Time', axis=1, inplace=True)
    return df_copy
```

Time 피처의 경우는 데이터 생성 관련한
작업용 속성으로서 큰 의미가 없기에 제거

```
# 사전 데이터 가공 후 학습과 테스트 데이터 세트를 반환하는 함수.
def get_train_test_dataset(df=None):
    # 인자로 입력된 DataFrame의 사전 데이터 가공이 완료된 복사 DataFrame 반환
    df_copy = get_preprocessed_df(df)
    # DataFrame의 맨 마지막 칼럼이 레이블, 나머지는 피쳐들
    X_features = df_copy.iloc[:, :-1]
    y_target = df_copy.iloc[:, -1]
    # train_test_split()으로 학습과 테스트 데이터 분할. stratify=y_target으로 Stratified 기반 분할
    X_train, X_test, y_train, y_test = \
        train_test_split(X_features, y_target, test_size=0.3, random_state=0, stratify=y_target)
    # 학습과 테스트 데이터 세트 반환
    return X_train, X_test, y_train, y_test
```

테스트 데이터 세트를 전체의 30%인 Stratified 방식으로 추출해
학습 데이터 세트와 테스트 데이터 세트의 레이블 값 분포도를 같게 함

```
X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)
```

get_train_test_dataset()는 get_preprocessed_clf()를 호출한 뒤
학습 피쳐/레이블 데이터 세트, 테스트 피쳐/레이블 데이터 세트를 반환

```
print('학습 데이터 레이블 값 비율')
print(y_train.value_counts()/y_train.shape[0] * 100)
print('테스트 데이터 레이블 값 비율')
print(y_test.value_counts()/y_test.shape[0] * 100)
```

```
학습 데이터 레이블 값 비율
0    99.827451
1     0.172549
테스트 데이터 레이블 값 비율
0    99.826785
1     0.173215
```

학습 데이터 레이블의 경우 1값이 약 0.172%, 테스트
데이터 레이블의 경우 1값이 약 0.173%로
큰 차이가 없이 잘 분할됨

#4.10 분류 실습 – 캐글 신용카드 사기 검출

< get_model_train_eval():모델을 변경해 학습/예측/평가할 수 있는 함수 생성 >

```
# 인자로 사이킷런의 Estimator 객체와 학습/테스트 데이터 세트를 입력받아서 학습/예측/평가 수행.
def get_model_train_eval(model, ftr_train=None, ftr_test=None, tgt_train=None, tgt_test=None):
    model.fit(ftr_train, tgt_train)
    pred = model.predict(ftr_test)
    pred_proba = model.predict_proba(ftr_test)[: , 1]
    get_clf_eval(tgt_test, pred, pred_proba)
```

get_model_train_eval()는 인자로 사이킷런의 Estimator 객체와 학습/테스트 데이터 세트를 입력받아서 학습/예측/평가를 수행하는 역할을 하는 함수

<로지스틱 회귀를 이용해 신용 카드 사기 여부 예측>

```
from sklearn.linear_model import LogisticRegression

lr_clf = LogisticRegression(max_iter=1000)
lr_clf.fit(X_train, y_train)
lr_pred = lr_clf.predict(X_test)
lr_pred_proba = lr_clf.predict_proba(X_test)[: , 1]

# 3장에서 사용한 get_clf_eval() 함수를 이용해 평가 수행.
get_clf_eval(y_test, lr_pred, lr_pred_proba)
```

오차 행렬
[[85281 14]
[56 92]]
정확도: 0.9992, 정밀도: 0.8679, 재현율: 0.6216, F1: 0.7244, AUC:0.9702

<Light GBM을 이용해 신용 카드 사기 여부 예측>

```
from lightgbm import LGBMClassifier

lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)
get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)
```

본 데이터 세트는 극도로 불균형한 레이블 값 분포도를 가지므로 boost_from_average=False로 파라미터 설정하여 초기 예측값을 전체 레이블 평균으로 하지 않음.

오차 행렬
[[85290 5]
[36 112]]
정확도: 0.9995, 정밀도: 0.9573, 재현율: 0.7568, F1: 0.8453, AUC:0.9790

로지스틱 회귀보다 Light GBM이 더 높은 수치 나타냄

#4.10 분류 실습 – 캐글 신용카드 사기 검출

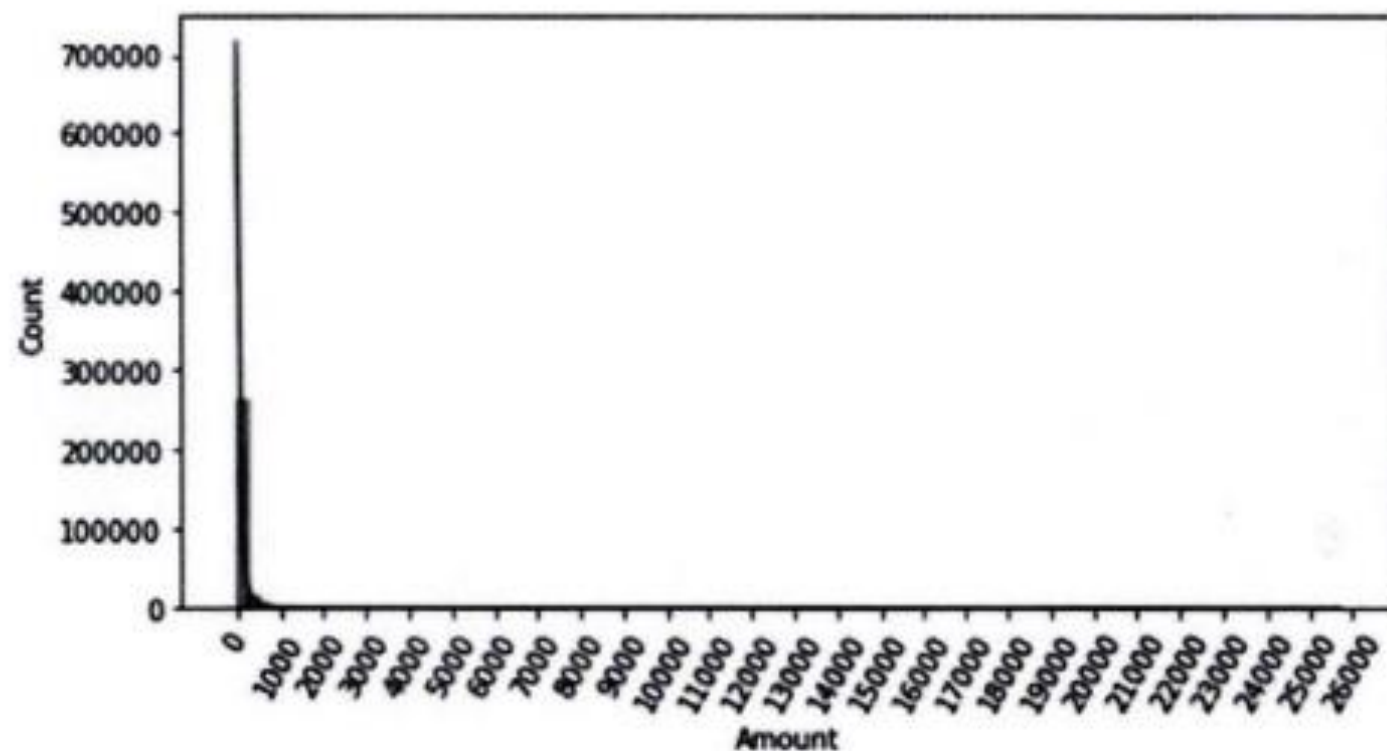
#5 데이터 분포도 변환 후 모델 학습/예측/평가 : 왜곡된 분포도를 가지는 데이터를 재가공한 후 모델을 다시 테스트

< Amount 피처의 분포도 확인>

신용 카드 사용 금액으로 정상/사기 트랜잭션을 결정하는 매우 중요한 속성일 가능성이 높음

```
import seaborn as sns

plt.figure(figsize=(8, 4))
plt.xticks(range(0, 30000, 1000), rotation=60)
sns.histplot(card_df['Amount'], bins=100, kde=True)
plt.show()
```



Amount, 즉 카드 사용금액이 1000불 이하인 데이터가 대부분이며, 26,000불까지만 드물지만 많은 금액을 사용한 경우가 발생하면서 꼬리가 긴 형태의 분포 곡선

1. 데이터를 정규 분포 형태로 재가공

get_preprocessed_df() 함수를 'Amount 피처를 정규 분포 형태로 변환하는 코드'로 변경 (사이킷런 **StandardScaler** 클래스)

-> 대부분의 선형 모델(로지스틱 회귀 포함)은 중요 피처들의 값이 정규 분포 형태를 유지하는 것을 선호하기 때문

```
from sklearn.preprocessing import StandardScaler
# 사이킷런의 StandardScaler를 이용해 정규 분포 형태로 Amount 피처값 변환하는 로직으로 수정.
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    scaler = StandardScaler()
    amount_n = scaler.fit_transform(df_copy['Amount'].values.reshape(-1, 1))
    # 변환된 Amount를 Amount_Scaled로 피처명 변경후 DataFrame맨 앞 칼럼으로 입력
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    # 기존 Time, Amount 피처 삭제
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    return df_copy
```

#4.10 분류 실습 – 캐글 신용카드 사기 검출

로지스틱 회귀와 LightGBM 모델을 각각 학습/예측/평가

```
# Amount를 정규 분포 형태로 변환 후 로지스틱 회귀 및 LightGBM 수행.
X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)

print('### 로지스틱 회귀 예측 성능 ###')
lr_clf = LogisticRegression(max_iter=10000)
get_model_train_eval(lr_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)

print('### LightGBM 예측 성능 ###')
lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)
get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)
```

```
### 로지스틱 회귀 예측 성능 ###
오차 행렬
[[85281  14]
 [  58  90]]
정확도: 0.9992, 정밀도: 0.8654, 재현율: 0.6081, F1: 0.7143, AUC:0.9702
### LightGBM 예측 성능 ###
오차 행렬
[[85290   5]
 [  37 111]]
정확도: 0.9995, 정밀도: 0.9569, 재현율: 0.7500, F1: 0.8409, AUC:0.9779
```

정규 분포 형태로 Amount 피쳐값을 변환한 후



로지스틱 회귀 -> 정밀도와 재현율이 오히려 조금 저하
LightGBM-> 약간 정밀도와 재현율이 저하

그러나 큰 성능상의 변화는 없음

#4.10 분류 실습 – 캐글 신용카드 사기 검출

2. 데이터를 로그변환하여 재가공

get_preprocessed_df() 함수를
'Amount 피처를 로그변환하는 코드'로 변경
(numpy `log1p()` 함수)

-> 원래 값을 log 값으로 변환해 원래 큰 값을 상대적으로 작은 값으로 변환하기 때문에
데이터 분포도의 왜곡을 상당 수준 개선

```
def get_preprocessed_df(df=None):  
    df_copy = df.copy()  
    # 넘파이의 log1p( )를 이용해 Amount를 로그 변환  
    amount_n = np.log1p(df_copy['Amount'])  
    df_copy.insert(0, 'Amount_Scaled', amount_n)  
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)  
    return df_copy
```

<로지스틱 회귀와 LightGBM 모델을 각각 학습/예측/평가

```
X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)  
  
print('### 로지스틱 회귀 예측 성능 ###')  
get_model_train_eval(lr_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train,  
                      tgt_test=y_test)  
  
print('### LightGBM 예측 성능 ###')  
get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train,  
                     tgt_test=y_test)
```

```
### 로지스틱 회귀 예측 성능 ###  
오차 행렬  
[[85283  12]  
 [  59  89]]  
정확도: 0.9992, 정밀도: 0.8812, 재현율: 0.6014, F1: 0.7149, AUC:0.9727  
### LightGBM 예측 성능 ###  
오차 행렬  
[[85290   5]  
 [  35 113]]  
정확도: 0.9995, 정밀도: 0.9576, 재현율: 0.7635, F1: 0.8496, AUC:0.9796
```

Amount 피처를 로그 변환한 후



- 로지스틱 회귀 -> 원본 데이터 대비 정밀도는 향상되었지만 재현율은 저하
- LightGBM-> 재현율 향상

레이블이 극도로 불균일한 데이터 세트에서 로지스틱 회귀는
데이터 변환 시 약간의 불안정한 성능 결과를 보여줌

#4.10 분류 실습 – 캐글 신용카드 사기 검출

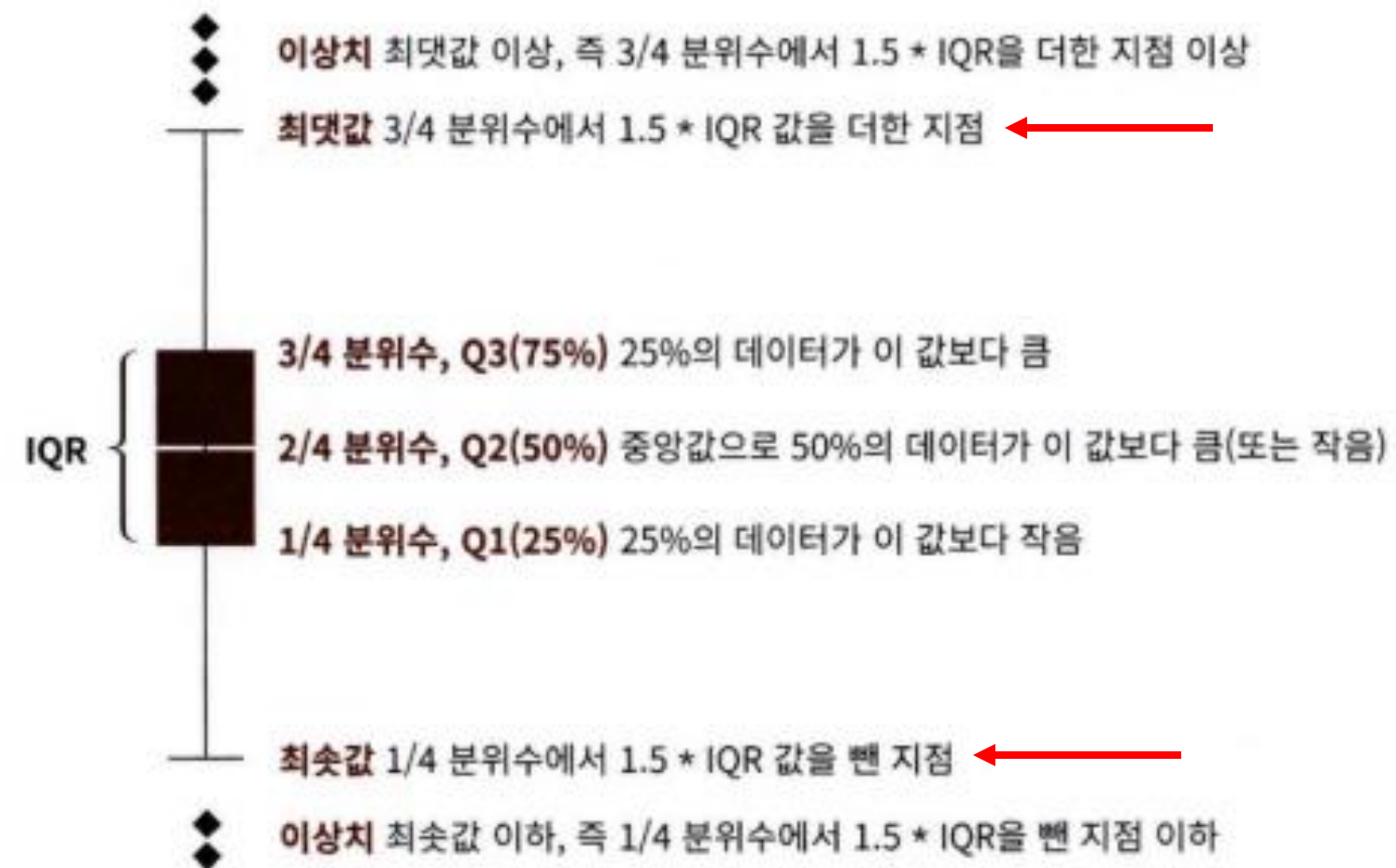
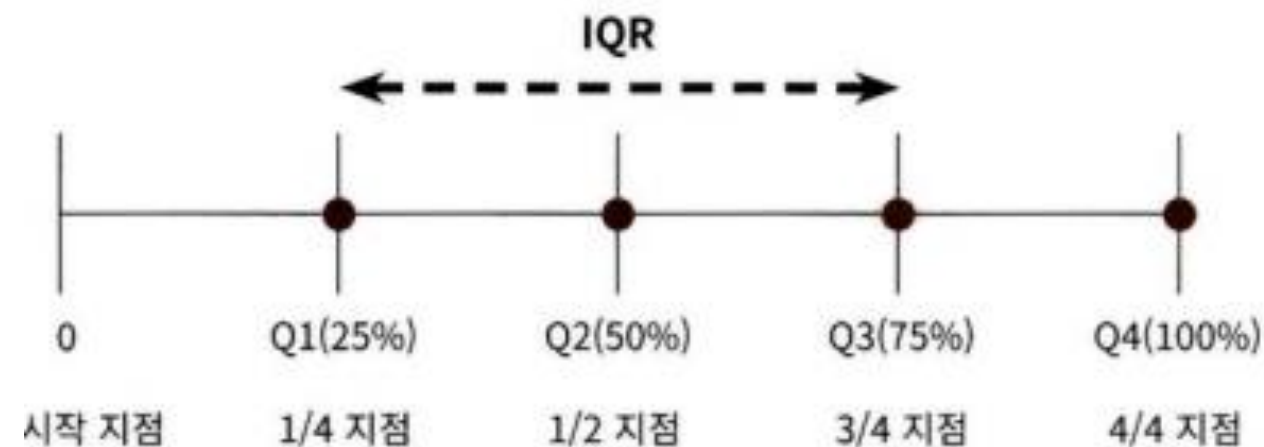
#6 이상치 데이터 제거 후 모델 학습/예측/평가

이상치 데이터, 아웃라이어(Outlier): 전체 데이터의 패턴에서 벗어난 이상 값을 가진 데이터

- 머신러닝 모델의 성능에 영향을 주기 쉬움
- 이상치를 찾아내고 이들 데이터를 제거할 필요가 있음

< 이상치 찾는 방법: IQR(Inter Quantile Range) 방식>

- IQR: 중 25% 구간인 Q1 ~ 75% 구간인 Q3의 범위
- 사분위: 전체 데이터를 값이 높은 순으로 정렬하고, 이를 1/4(25%) 씩으로 구간을 분할하는 것



이렇게 결정된 최댓값보다 큰 값 또는 최솟값보다 작은 값을 이상치 데이터로 간주

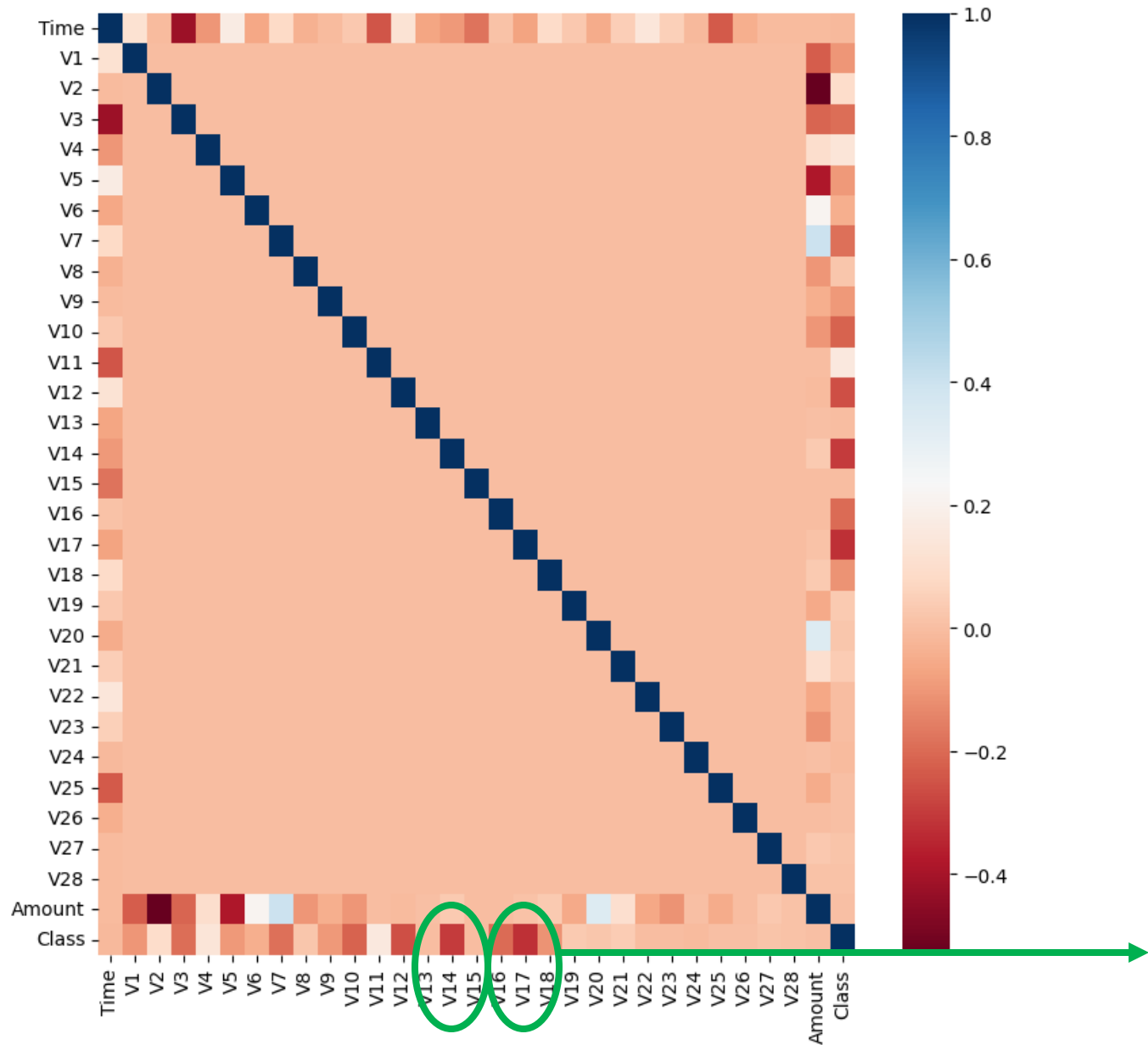
#4.10 분류 실습 – 캐글 신용카드 사기 검출

어떤 피처의 이상치 데이터를 검출할지 선택

→ DataFrame의 corr()을 이용해 각 피처별로 상관을 구하여 결정 레이블과 상관관계가 높은 피처 선택

```
import seaborn as sns

plt.figure(figsize=(9, 9))
corr = card_df.corr()
sns.heatmap(corr, cmap='RdBu')
```



상관관계 히트맵에서 cmap을 'RdBu'로 설정
→ 양의 상관관계가 높을수록 색깔이 진한 파란색에 가까움
→ 음의 상관관계가 높을수록 색깔이 진한 빨간색에 가까움

상관관계 히트맵에서 맨 아래에 위치한 결정 레이블인 Class 피처와 음의 상관관계가 가장 높은 피처는 V14와 V17

#4.10 분류 실습 – 캐글 신용카드 사기 검출

IQR을 이용해 이상치를 검출하는 함수를 생성 한 뒤, 이를 이용해 검출된 **이상치를 삭제**

1. 이상치를 검출하는 get_outlier() 함수 생성

```
import numpy as np

def get_outlier(df=None, column=None, weight=1.5):
    # fraud에 해당하는 column 데이터만 추출, 1/4 분위와 3/4 분위 지점을 np.percentile로 구함.
    fraud = df[df['Class']==1][column]
    quantile_25 = np.percentile(fraud.values, 25)
    quantile_75 = np.percentile(fraud.values, 75)
    # IQR을 구하고, IQR에 1.5를 곱해 최댓값과 최솟값 지점 구함.
    iqr = quantile_75 - quantile_25
    iqr_weight = iqr * weight
    lowest_val = quantile_25 - iqr_weight
    highest_val = quantile_75 + iqr_weight
    # 최댓값보다 크거나, 최솟값보다 작은 값을 이상치 데이터로 설정하고 DataFrame index 반환.
    outlier_index = fraud[(fraud < lowest_val) | (fraud > highest_val)].index
    return outlier_index
```

get_outlier() 함수는 인자로 Dataframe과 이상치 를 검출한 칼럼을 입력받음

2. get_outlier() 함수를 이용해 V14 칼럼에서 이상치 데이터를 찾기

```
outlier_index = get_outlier(df=card_df, column='V14', weight=1.5)
print('이상치 데이터 인덱스:', outlier_index)
```

이상치 데이터 인덱스: Int64Index([8296, 8615, 9035, 9252], dtype='int64')

3. get_outlier()를 이용해 이상치를 추출하고 이를 삭제하는 로직을 get_preprocessed_df() 함수에 추가해 데이터 가공

```
# get_preprocessed_df()를 로그 변환 후 V14 피쳐의 이상치 데이터를 삭제하는 로직으로 변경.
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    amount_n = np.log1p(df_copy['Amount'])
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    # 이상치 데이터 삭제하는 로직 추가
    outlier_index = get_outlier(df=df_copy, column='V14', weight=1.5)
    df_copy.drop(outlier_index, axis=0, inplace=True)
    return df_copy
```

4. 가공된 데이터 세트를 이용해 로지스틱 회귀와 LightGBM 모델을 다시 적용

```
X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)
print('### 로지스틱 회귀 예측 성능 ###')
get_model_train_eval(lr_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train,
                    tgt_test=y_test)
print('### LightGBM 예측 성능 ###')
get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)
```

로지스틱 회귀 예측 성능

오차 행렬

```
[[85281  14]
 [  48  98]]
```

정확도: 0.9993, 정밀도: 0.8750, 재현율: 0.6712, F1: 0.7597, AUC:0.9743

LightGBM 예측 성능

오차 행렬

```
[[85290   5]
 [  25 121]]
```

정확도: 0.9996, 정밀도: 0.9603, 재현율: 0.8288, F1: 0.8897, AUC:0.9780

→ 이상치 데이터를 제거한 뒤, 로지스틱 회귀와 LightGBM 모두 예측 성능이 크게 향상

#4.10 분류 실습 – 캐글 신용카드 사기 검출

#7 SMOTE 오버 샘플링 적용 후 모델 학습/예측/평가

SMOTE를 적용할 때는 반드시 학습 데이터 세트만 오버 샘플링

→ 검증 데이터 세트나 테스트 데이터 세트를 오버 샘플링할 경우 원본 데이터 세트가 아닌 데이터 세트에서 검증 또는 테스트를 수행하기 때문에 올바른 검증/테스트가 될 수 없음

<SMOTE 기법으로 오버 샘플링을 적용한 뒤 로지스틱 회귀모델의 예측 성능을 평가>

1. 학습 피쳐/레이블 데이터를 SMOTE 객체의 `fit_resample()` 메서드를 이용해 증식>

```
from imblearn.over_sampling import SMOTE
```

 SMOTE: imbalanced-learn 패키지의 SMOTE 클래스를 이용

```
smote = SMOTE(random_state=0)
X_train_over, y_train_over = smote.fit_resample(X_train, y_train)
print('SMOTE 적용 전 학습용 피쳐/레이블 데이터 세트: ', X_train.shape, y_train.shape)
print('SMOTE 적용 후 학습용 피쳐/레이블 데이터 세트: ', X_train_over.shape, y_train_over.shape)
print('SMOTE 적용 후 레이블 값 분포: \n', pd.Series(y_train_over).value_counts())
```

```
SMOTE 적용 전 학습용 피쳐/레이블 데이터 세트: (199362, 29) (199362, )
SMOTE 적용 후 학습용 피쳐/레이블 데이터 세트: (398040, 29) (398040, )
SMOTE 적용 후 레이블 값 분포:
1    199020
0    199020
```

- SMOTE 적용 전 학습 데이터 세트는 199,362건이었지만 SMOTE 적용 후 2배에 가까운 398,040건으로 데이터가 증식됨
- SMOTE 적용 후 레이블 값이 0과 1 의 분포가 동일하게 199,020 건 으로 생성

#4.10 분류 실습 – 캐글 신용카드 사기 검출

2. 생성된 학습 데이터 세트를 기반으로 로지스틱 회귀 모델을 학습 한 뒤 성능 평가

```
lr_clf = LogisticRegression(max_iter=1000)
# ftr_train과 tgt_train 인자값이 SMOTE 증식된 X_train_over와 y_train_over로 변경됨에 유의
get_model_train_eval(lr_clf, ftr_train=X_train_over, ftr_test=X_test, tgt_train=y_train_over,
                    tgt_test=y_test)
```

오차 행렬

[[82937 2358]

[11 135]]

정확도: 0.9723, 정밀도: 0.0542, 재현율: 0.9247, F1: 0.1023, AUC:0.9737

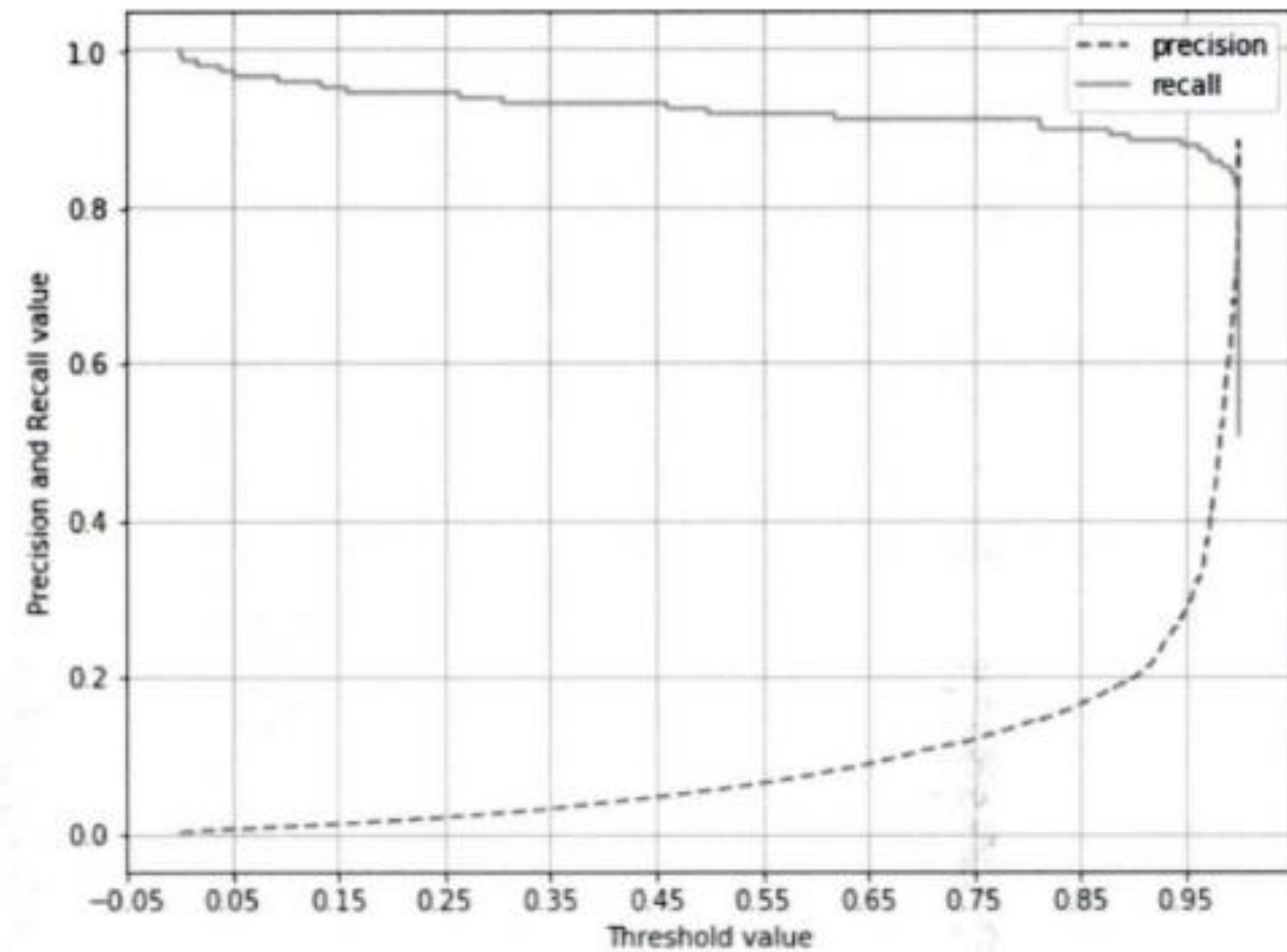
재현율이 92.47%로 크게 증가하지만, 반대로 정밀도가 5.4%로 급격하게 저하

→ 로지스틱 회귀 모델이 오버 샘플링으로 인해
실제 원본 데이터의 유형보다 너무나 많은 Class=1 데이터를 학습하면서
실제 테스트 데이터 세트에서 예측을 지나치게 Class=1로 적용해
정밀도가 급격히 떨어졌기 때문

3. 분류 결정 임계값에 따른 정밀도와 재현율 곡선

→ SMOTE로 학습된 로지스틱 회귀 모델에 어떠한 문제가 발생하고 있는지 확인

```
precision_recall_curve_plot( y_test, lr_clf.predict_proba(X_test)[: , 1] )
```



임계값 0.99 이하

→ 재현율이 매우 좋고 정밀도가 극단적으로 낮음

임계값 0.99 이상

→ 반대로 재현율이 대폭 떨어지고 정밀도가 높아짐

분류 결정 임계값을 조정하더라도 임계값의 민감도가 너무 심해 올바른 재현율/정밀도 성능을 얻을 수 없음.

로지스틱 회귀 모델의 경우 SMOTE 적용 후 올바른 예측 모델이 생성되지 못 함

#4.10 분류 실습 – 캐글 신용카드 사기 검출

〈SMOTE 기법으로 오버 샘플링을 적용한 뒤 *LightGBM* 모델의 예측 성능을 평가〉

```
lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)
get_model_train_eval(lgbm_clf, ftr_train=X_train_over, ftr_test=X_test,
                    tgt_train=y_train_over, tgt_test=y_test)
```

오차 행렬

```
[[85283  12]
 [  22 124]]
```

정확도: 0.9996, 정밀도: 0.9118, 재현율: 0.8493, F1: 0.8794, AUC:0.9814

SMOTE를 적용하면 재현율은 높아지나, 정밀도는 낮아지는 것이 일반적
좋은 SMOTE 패키지일수록 **재현율 증가율은 높이고 정밀도 감소율은 낮출** 수 있도록 효과적으로 데이터를 증식

재현율이 이상치만 제거한 경우인 82.88%보다 높은 84.93%

정밀도는 이전의 96.03%보다 낮은 91.18%

〈여러 방법으로 데이터를 가공하면서 로지스틱 회귀와 LightGBM을 적용한 결과〉

데이터 가공 유형	머신러닝 알고리즘	평가 지표		
		정밀도	재현율	ROC-AUC
원본 데이터 가공 없음	로지스틱 회귀	0.8679	0.6216	0.9702
	LightGBM	0.9573	0.7568	0.9790
데이터 로그 변환	로지스틱 회귀	0.8812	0.6014	0.9727
	LightGBM	0.9576	0.7635	0.9796
이상치 데이터 제거	로지스틱 회귀	0.8750	0.6712	0.9743
	LightGBM	0.9603	0.8288	0.9780
SMOTE 오버 샘플링	로지스틱 회귀	0.0542	0.9247	0.9737
	LightGBM	0.9118	0.8493	0.9814

11. 스테킹 앙상블



#4.11 스택킹 앙상블

#1 스택킹(Stacking)

:개별적인 여러 알고리즘을 서로 결합해 예측 결과를 도출(배깅(Bagging) 및 부스팅(Boosting)과 공통점)

- 배깅:

여러 개의 약한 학습기(주로 결정 트리)를 병렬로 학습.
데이터 샘플을 중복 허용 랜덤 샘플링으로 나눠 학습
Ex) 랜덤포레스트

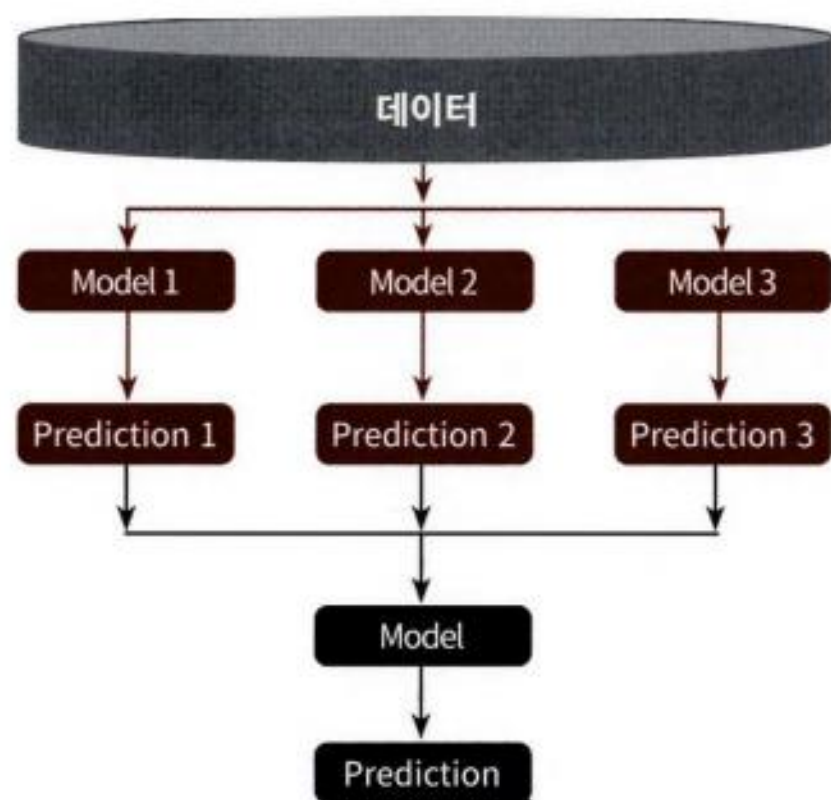
- 부스팅:

여러 학습기를 순차적으로 학습
이전 학습기가 잘못 예측한 데이터에 가중치를 높여 다음 학습기가 학습.
최종 예측은 각 모델의 가중합(weighted sum)
Ex) XGBoost, LightGBM

스택킹의 차이점: 개별 알고리즘으로 예측한 데이터를 기반으로 다시 예측을 수행한다는 것

개별 알고리즘의 예측 결과 데이터 세트를 최종적인 메타 데이터 세트로 만들어 별도의 ML 알고리즘으로 최종 학습을 수행하고 테스트 데이터를 기반으로 다시 최종 예측을 수행하는 방식

→ 이렇게 개별 모델의 예측된 데이터 세트를 다시 기반으로 하여 학습하고 예측하는 방식: 메타 모델



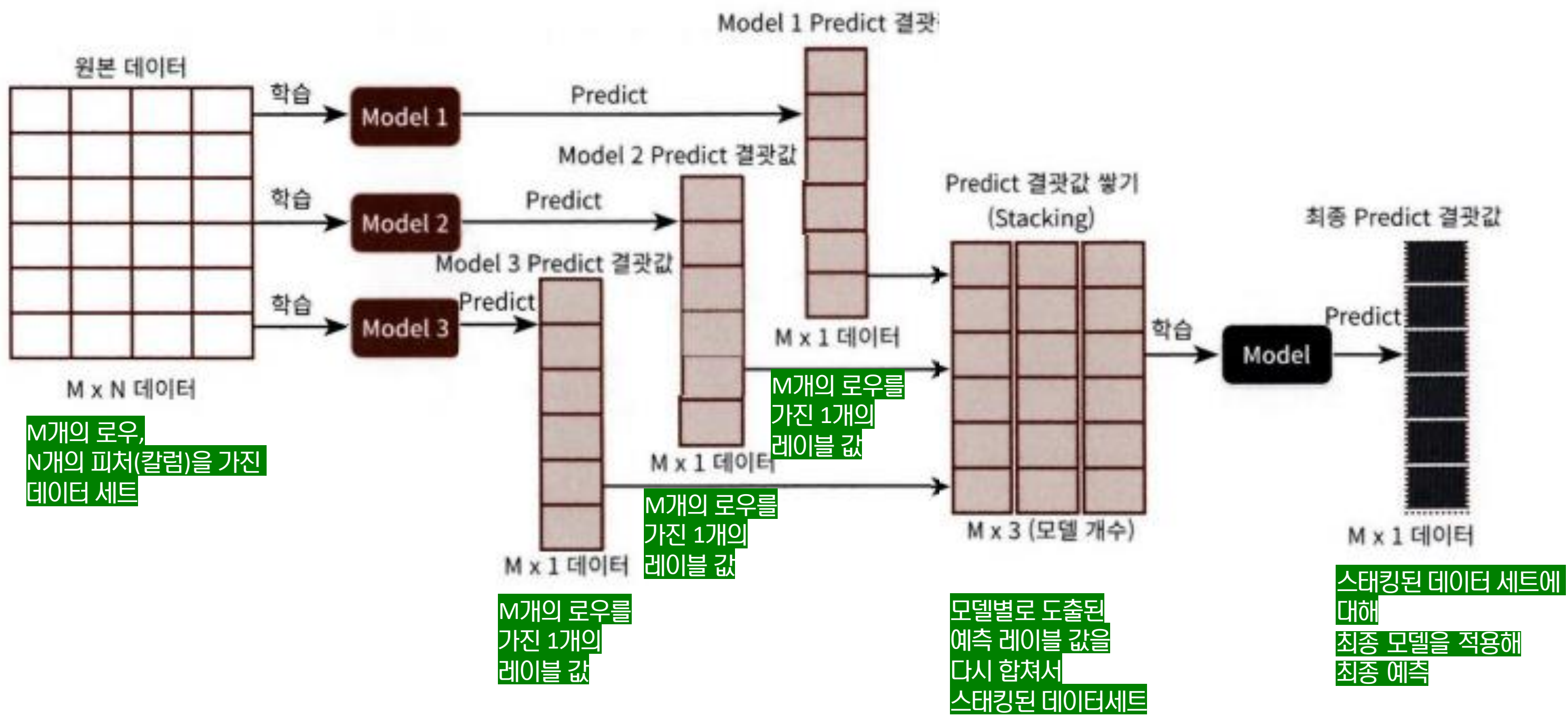
스택킹 모델은 두 종류의 모델이 필요.

→ 1. 개별적인 기반 모델

→ 2. 개별 기반 모델의 예측데이터를 만들어서 학습하는 최종 메타 모델

핵심: 여러 개별 모델의 예측 데이터를 각각 스택킹 형태로 결합해 최종 메타 모델의 학습용 피쳐 데이터 세트와 테스트용 피쳐 데이터 세트를 만드는 것

#4.11 스택킹 앙상블



#4.11 스택킹 앙상블

2 기본 스택킹 모델

기본 스택킹 모델을 위스콘신 암 데이터 세트에 적용
데이터 로딩하고 학습 데이터 세트와 테스트 데이터 세트로 나눔

```
import numpy as np

from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

cancer_data = load_breast_cancer()

X_data = cancer_data.data
y_label = cancer_data.target

X_train, X_test, y_train, y_test = train_test_split(X_data, y_label, test_size=0.2, random_state=0)
```


#4.11 스타킹 앙상블

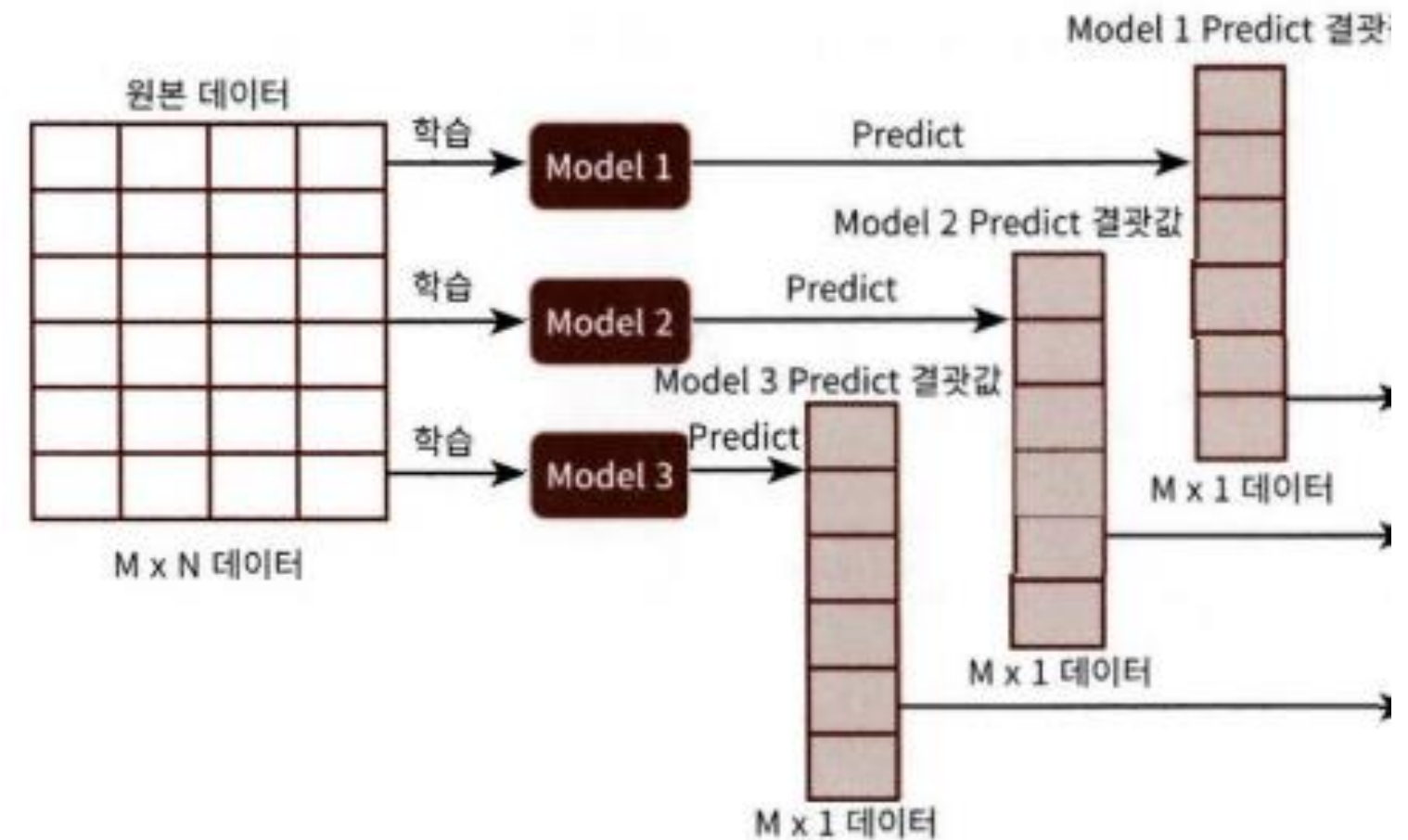
```
# 개별 ML 모델 생성
knn_clf = KNeighborsClassifier(n_neighbors=4)
rf_clf = RandomForestClassifier(n_estimators=100, random_state=0)
dt_clf = DecisionTreeClassifier()
ada_clf = AdaBoostClassifier(n_estimators=100)

# 스타킹으로 만들어진 데이터 세트를 학습, 예측할 최종 모델
lr_final = LogisticRegression()
```

```
# 개별 모델들을 학습.
knn_clf.fit(X_train, y_train)
rf_clf.fit(X_train, y_train)
dt_clf.fit(X_train, y_train)
ada_clf.fit(X_train, y_train)
```

```
# 학습된 개별 모델들이 각자 반환하는 예측 데이터 세트를 생성하고 개별 모델의 정확도 측정.
knn_pred = knn_clf.predict(X_test)
rf_pred = rf_clf.predict(X_test)
dt_pred = dt_clf.predict(X_test)
ada_pred = ada_clf.predict(X_test)
print('KNN 정확도: {0:.4f}'.format(accuracy_score(y_test, knn_pred)))
print('랜덤 포레스트 정확도: {0:.4f}'.format(accuracy_score(y_test, rf_pred)))
print('결정 트리 정확도: {0:.4f}'.format(accuracy_score(y_test, dt_pred)))
print('에이다부스트 정확도: {0:.4f}'.format(accuracy_score(y_test, ada_pred)))
```

<개별 모델들이 예측 데이터 세트 생성>



KNN 정확도: 0.9211
랜덤 포레스트 정확도: 0.9649
결정 트리 정확도: 0.9123
에이다부스트 정확도: 0.9561

#4.11 스택킹 앙상블

```
pred = np.array([knn_pred, rf_pred, dt_pred, ada_pred])
print(pred.shape)

# transpose를 이용해 행과 열의 위치 교환. 칼럼 레벨로 각 알고리즘의 예측 결과를 피처로 만들.
pred = np.transpose(pred)
print(pred.shape)
```

(4, 114)
(114, 4)

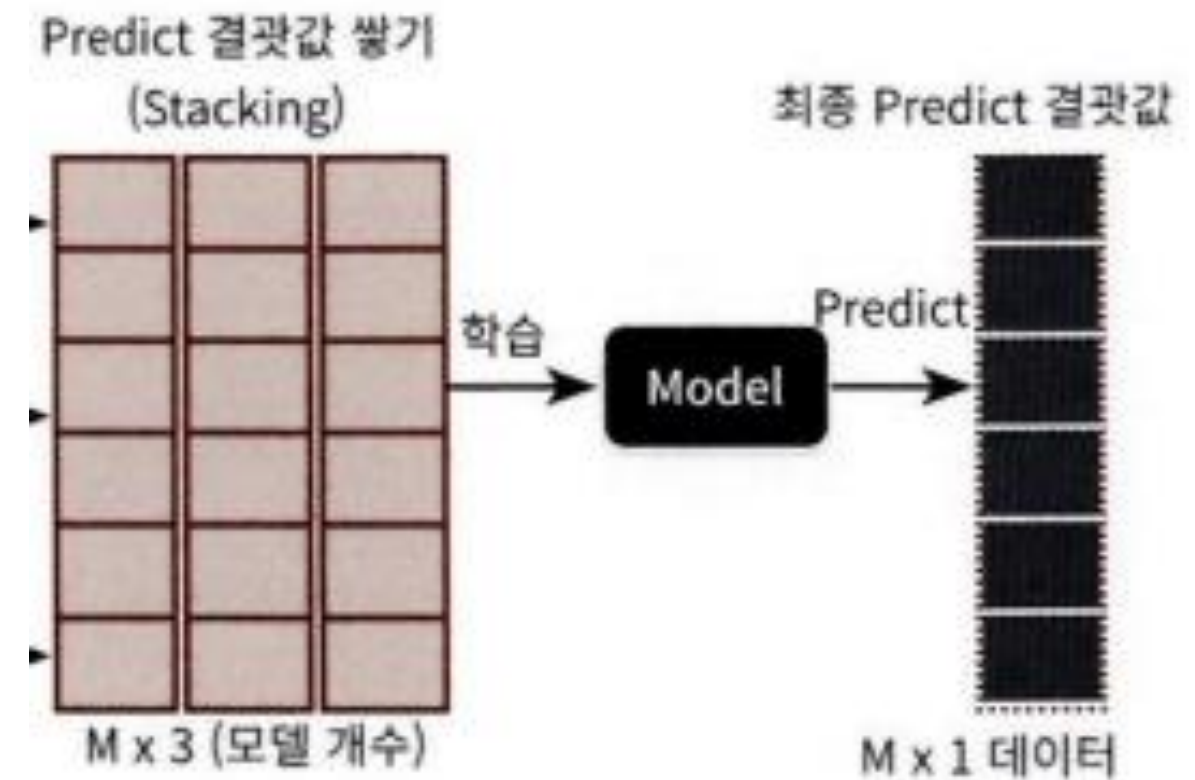
```
lr_final.fit(pred, y_test)
final = lr_final.predict(pred)

print('최종 메타 모델의 예측 정확도: {0:.4f}'.format(accuracy_score(y_test, final)))
```

최종 메타 모델의 예측 정확도: 0.9737

개별 모델의 예측 데이터를 스택킹으로 재구성해 최종 메타 모델에서 학습하고 예측한 결과, 정확도가 97.37%로 개별 모델 정확도보다 향상됨
(이러한 스택킹 기법으로 예측을 한다고 무조건 개별 모델보다는 좋아진다는 보장은 없음)

<예측 데이터로 생성된 데이터 세트를 기반으로 최종 메타 모델인 로지스틱 회귀를 학습>



#4.11 스택킹 앙상블

#3 CV(Cross Validation) 세트 기반의 스택킹

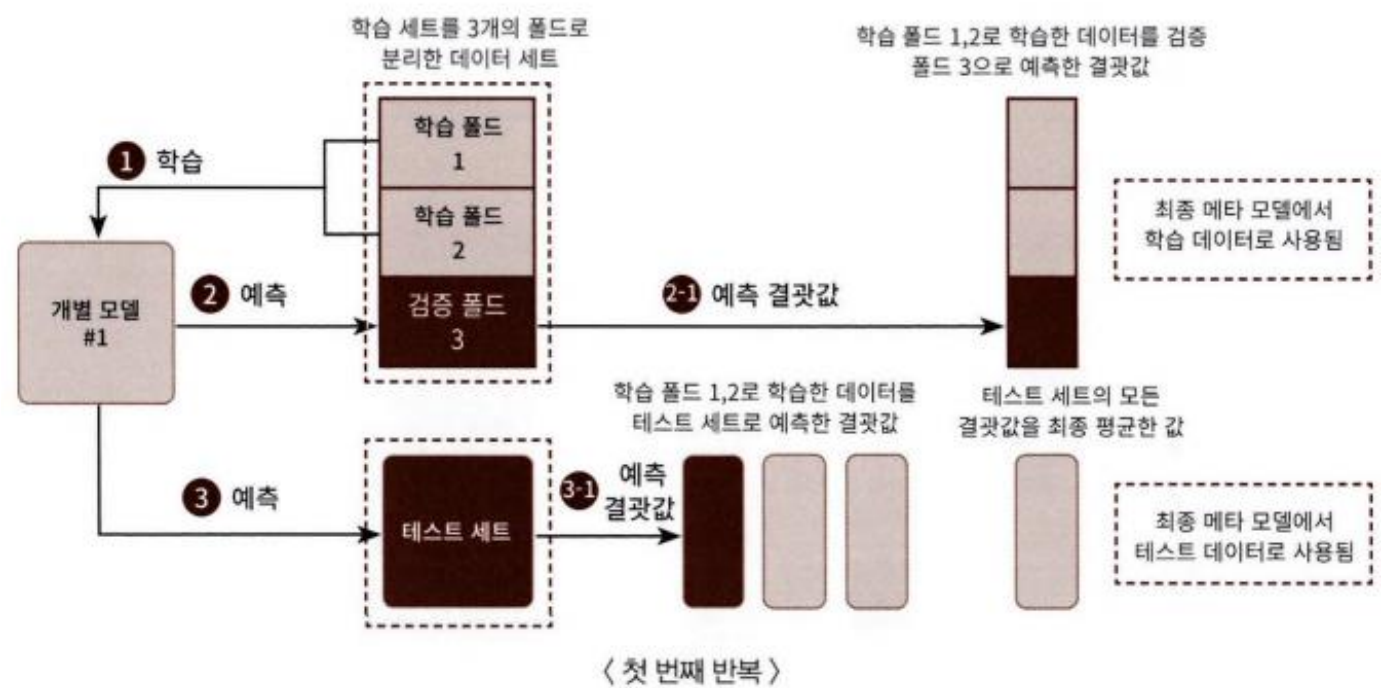
: 과적합을 개선하기 위해 최종 메타 모델을 위한 데이터 세트를 만들 때 교차 검증 기반으로 예측된 결과 데이터 세트를 이용
개별 모델들이 각각 교차 검증으로 **메타 모델을 위한 학습용 스택킹 데이터** 생성과 **예측을 위한 테스트용 스택킹 데이터**를 생성한 뒤
이를 기반으로 메타 모델이 학습과 예측 수행

<스텝1 - 개별 모델>

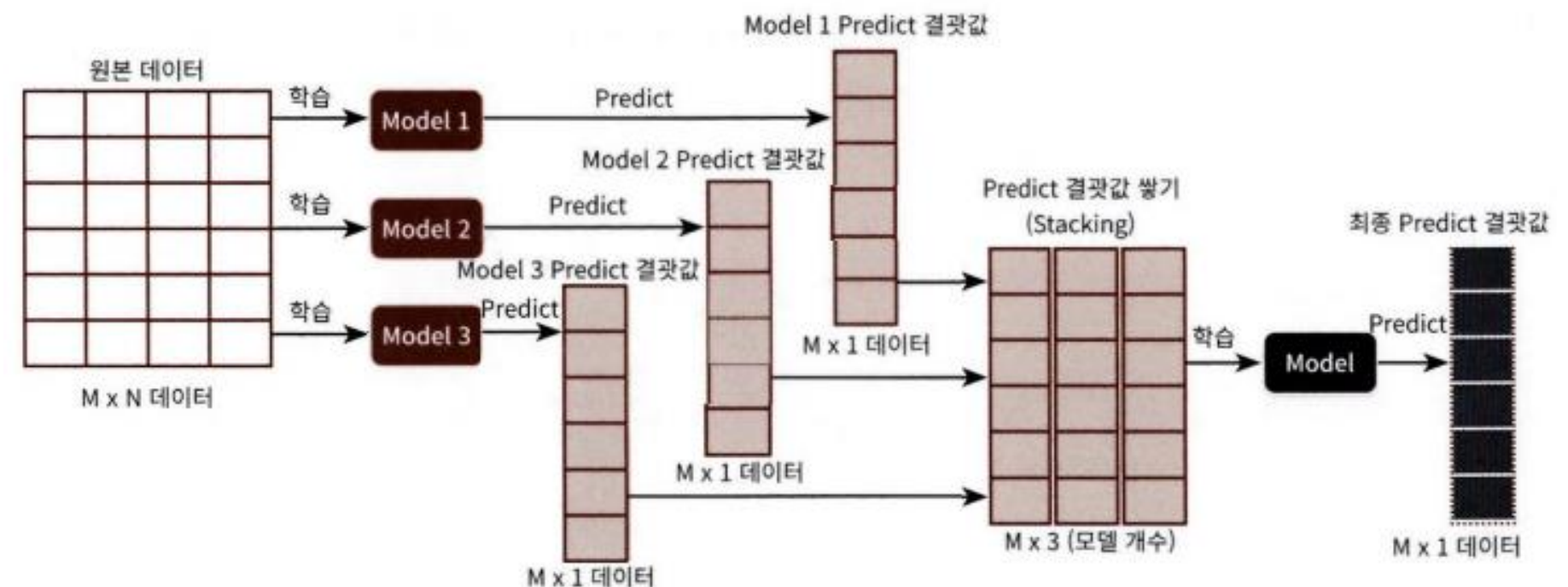
각 모델별로 원본 학습/테스트 데이터를 예측한 결과 값을 기반으로 **메타 모델을 위한 학습용/테스트용 데이터를 생성**

<스텝2 - 메타 모델>

스텝 1에서 개별 모델들이 생성한 학습용 데이터를 모두 스택킹 형태로 합쳐서 **메타 모델이 학습할 최종 학습용 데이터 세트**를 생성
마찬가지로 각 모델들이 생성한 테스트용 데이터를 모두 스택킹 형태로 합쳐서 **메타 모델이 예측할 최종 테스트 데이터 세트**를 생성
메타 모델은 최종적으로 생성된 학습 데이터 세트와 원본 학습 데이터의 레이블 데이터를 기반으로 학습한 뒤,
최종적으로 생성된 테스트 데이터 세트를 예측하고, **원본 테스트 데이터의 레이블 데이터를 기반으로 평가**



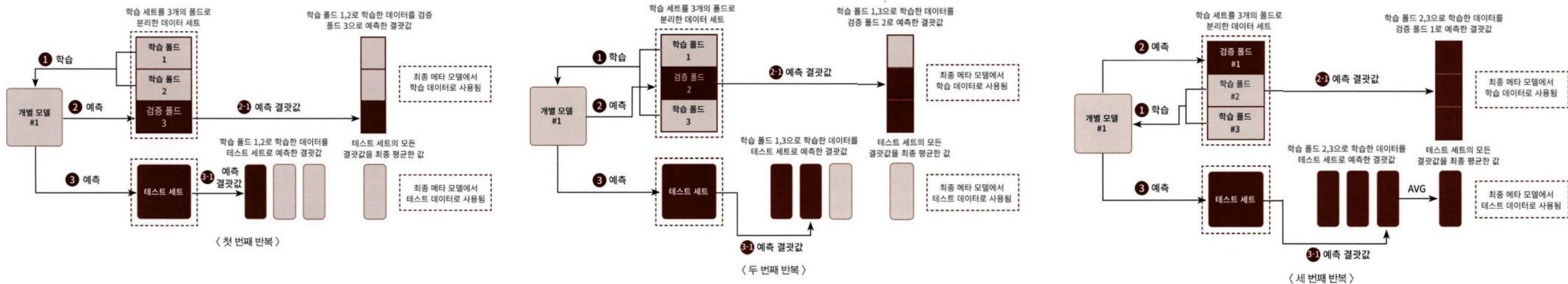
CV 세트 기반의 스택킹



과적합 문제가 발생할 수 있는 스택킹

#4.11 스타킹 앙상블

<스텝1 - 개별 모델에서 수행>



개별 모델에서 메타 모델인 2차 모델에서 사용될 학습용 데이터와 테스트 데이터를 교차 검증을 통해서 생성

1. 학습용 데이터의 N개의 폴드(Fold)로 나눔(예시: 3개의 폴드세트)
2개의 폴드는 **학습을 위한 데이터 폴드**, 나머지 1개는 **검증을 위한 데이터 폴드**로 나눔
두 개의 폴드로 나뉜 학습 데이터를 기반으로 개별 모델을 학습시킴

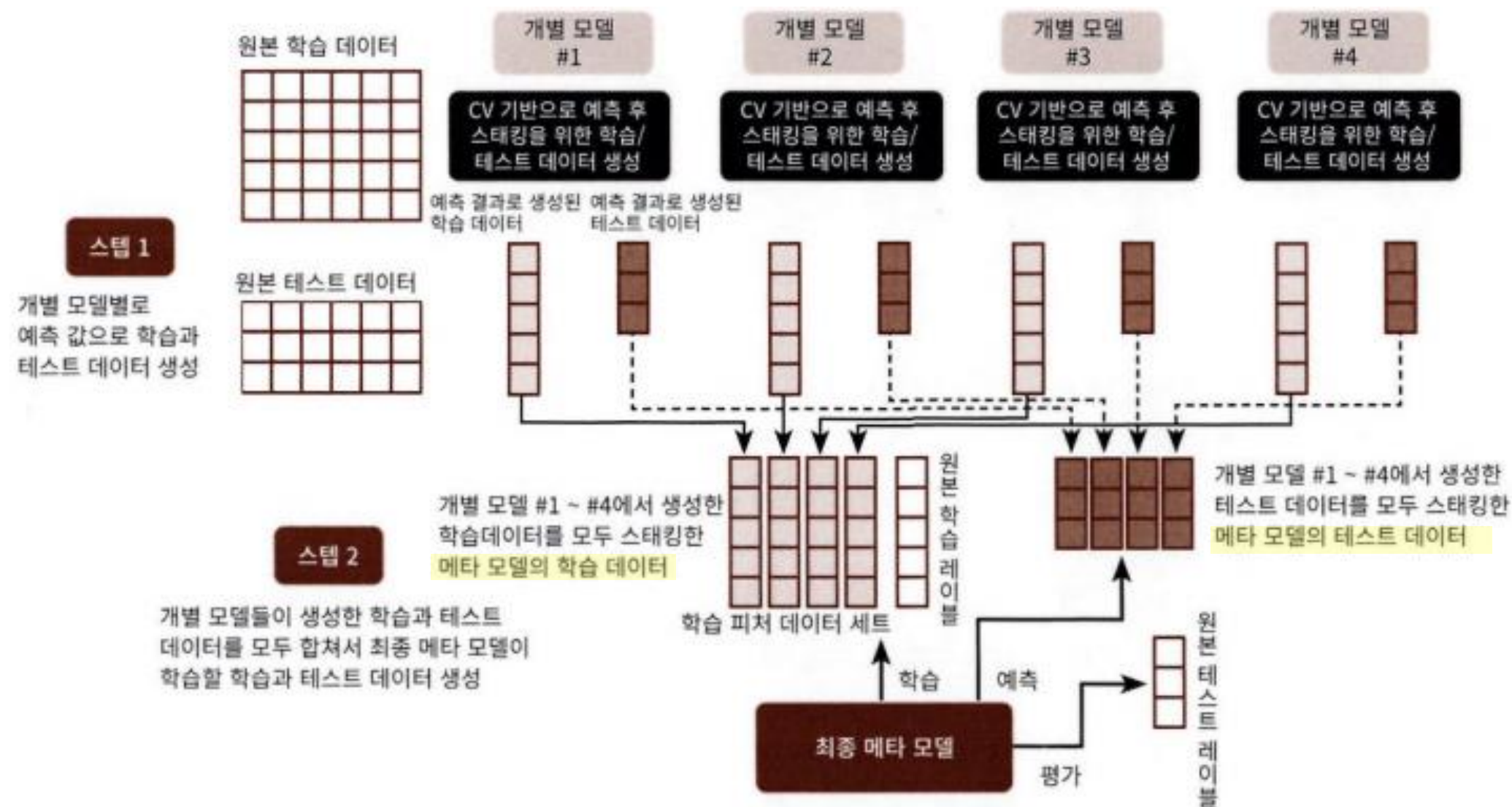
2. 학습된 개별모델은 검증 폴드 1개 데이터로 예측하고 그 결과 저장
이러한 로직을 학습데이터와 검증 데이터 세트를 변경하며 3번 반복
이렇게 만들어진 예측 데이터는 **메타 모델을 학습시키는 학습데이터**로 이용

3. 2개의 학습 폴드 데이터로 학습된 개별 모델은 **원본 테스트 데이터를 예측**하여 예측값을 생성.
이러한 로직을 3번 반복하면서 이 예측값의 평균으로 최종 결괏값을 생성하고 이를 **메타 모델을 위한 테스트 데이터**로 사용

반복을 모두 완료하면 반복을 수행하면서 만들어진 폴드별 예측 데이터를 합하여 메타 모델에서 사용될 학습 데이터 그리고 폴드 데이터로 학습된 개별 모델이 원본 테스트 세트로 예측한 결괏값을 최종 평균하여 메타 모델에서 사용될 테스트 데이터를 만들

#4.11 스택킹 앙상블

<스텝2 - 메타 모델을 학습>



1. 각 모델들이 스텝 1로 생성한 학습과 테스트 데이터를 모두 합쳐서 최종적으로 메타 모델이 사용할 학습 데이터와 테스트 데이터를 생성
2. 메타 모델이 사용할 최종 학습 데이터와 원본 데이터의 레이블 데이터를 합쳐서 메타 모델을 학습
3. 최종 테스트 데이터로 예측을 수행
4. 최종 예측 결과를 원본 테스트 데이터의 레이블 데이터와 비교해 평가

#4.11 스택킹 앙상블

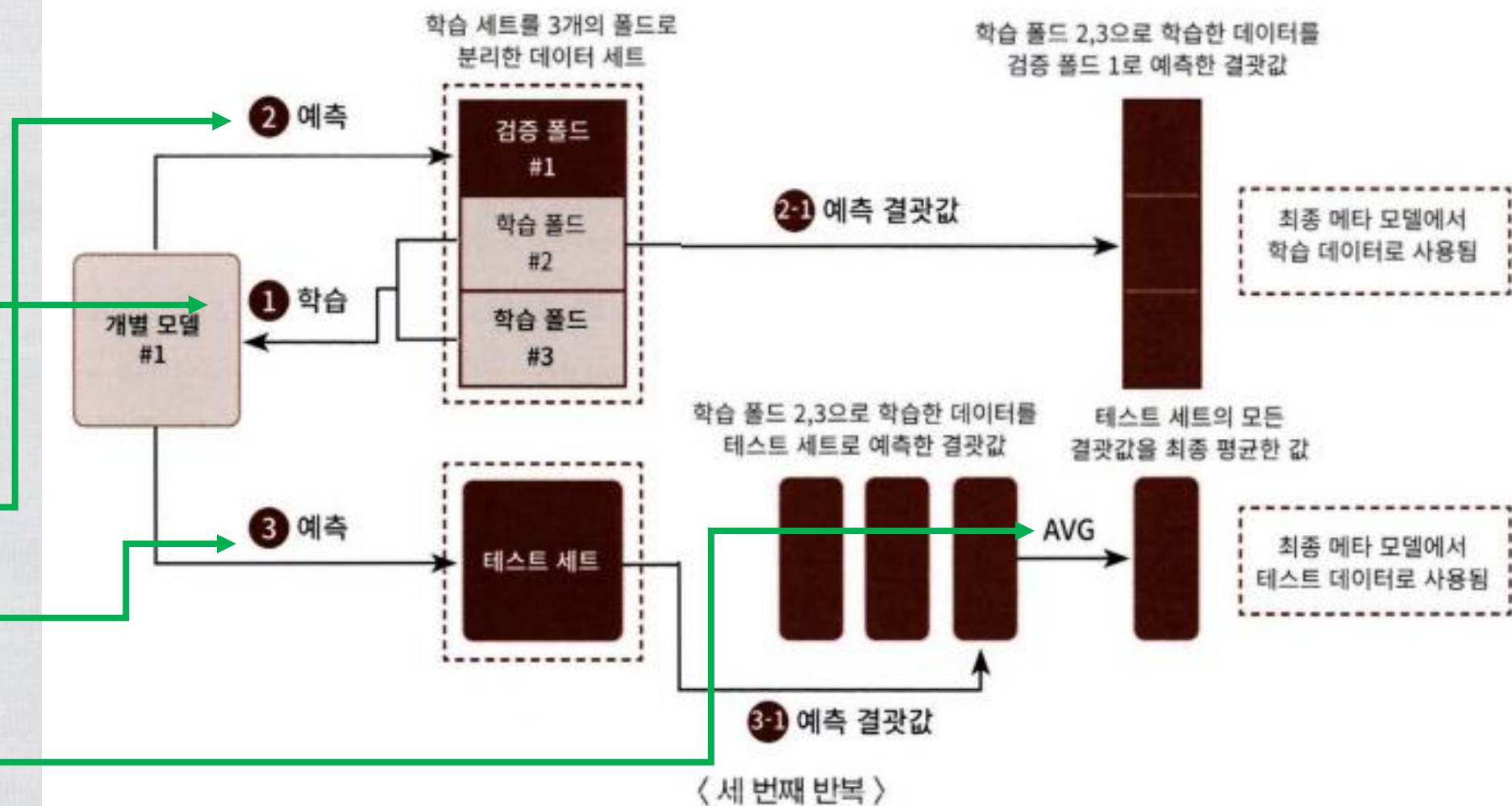
스텝1 코드

개별 기반 모델에서 최종 메타 모델이 사용할 학습 및 테스트용 데이터를 생성하기 위한 함수.

```
def get_stacking_base_datasets(model, X_train_n, y_train_n, X_test_n, n_folds):  
    # 지정된 n_folds값으로 KFold 생성.  
    kf = KFold(n_splits=n_folds, shuffle=False)  
    # 추후에 메타 모델이 사용할 학습 데이터 반환을 위한 넘파이 배열 초기화  
    train_fold_pred = np.zeros((X_train_n.shape[0], 1))  
    test_pred = np.zeros((X_test_n.shape[0], n_folds))  
    print(model.__class__.__name__, ' model 시작 ')  
    for folder_counter, (train_index, valid_index) in enumerate(kf.split(X_train_n)):  
        # 입력된 학습 데이터에서 기반 모델이 학습/예측할 폴드 데이터 세트 추출  
        print('\t 폴드 세트: ', folder_counter, ' 시작 ')  
        X_tr = X_train_n[train_index]  
        y_tr = y_train_n[train_index]  
        X_te = X_train_n[valid_index]  
  
        # 폴드 세트 내부에서 다시 만들어진 학습 데이터로 기반 모델의 학습 수행.  
        model.fit(X_tr, y_tr)  
        # 폴드 세트 내부에서 다시 만들어진 검증 데이터로 기반 모델 예측 후 데이터 저장.  
        train_fold_pred[valid_index, :] = model.predict(X_te).reshape(-1, 1)  
        # 입력된 원본 테스트 데이터를 폴드 세트내 학습된 기반 모델에서 예측 후 데이터 저장.  
        test_pred[:, folder_counter] = model.predict(X_test_n)  
  
    # 폴드 세트 내에서 원본 테스트 데이터를 예측한 데이터를 평균하여 테스트 데이터로 생성  
    test_pred_mean = np.mean(test_pred, axis=1).reshape(-1, 1)  
  
    #train_fold_pred는 최종 메타 모델이 사용하는 학습 데이터, test_pred_mean은 테스트 데이터  
    return train_fold_pred, test_pred_mean
```

```
knn_train, knn_test = get_stacking_base_datasets(knn_clf, X_train, y_train, X_test, 7)  
rf_train, rf_test = get_stacking_base_datasets(rf_clf, X_train, y_train, X_test, 7)  
dt_train, dt_test = get_stacking_base_datasets(dt_clf, X_train, y_train, X_test, 7)  
ada_train, ada_test = get_stacking_base_datasets(ada_clf, X_train, y_train, X_test, 7)
```

여러 개의 분류 모델별로 `stacking_base_model()` 함수를 수행
각각 메타 모델이 추후에 사용할 학습용, 테스트용 데이터 세트를 반환



#4.11 스택킹 앙상블

스텝2 코드

```
Stack_final_X_train = np.concatenate((knn_train, rf_train, dt_train, ada_train), axis=1)
Stack_final_X_test = np.concatenate((knn_test, rf_test, dt_test, ada_test), axis=1)
print('원본 학습 피쳐 데이터 Shape:', X_train.shape, '원본 테스트 피쳐 Shape:', X_test.shape)
print('스태킹 학습 피쳐 데이터 Shape:', Stack_final_X_train.shape,
      '스태킹 테스트 피쳐 데이터 Shape:', Stack_final_X_test.shape)
```

get_stacking_base_datasets() 호출로 반환된 각 모델 별 학습 데이터와 테스트 데이터를 numpy의 concatenate()를 이용해 합침.

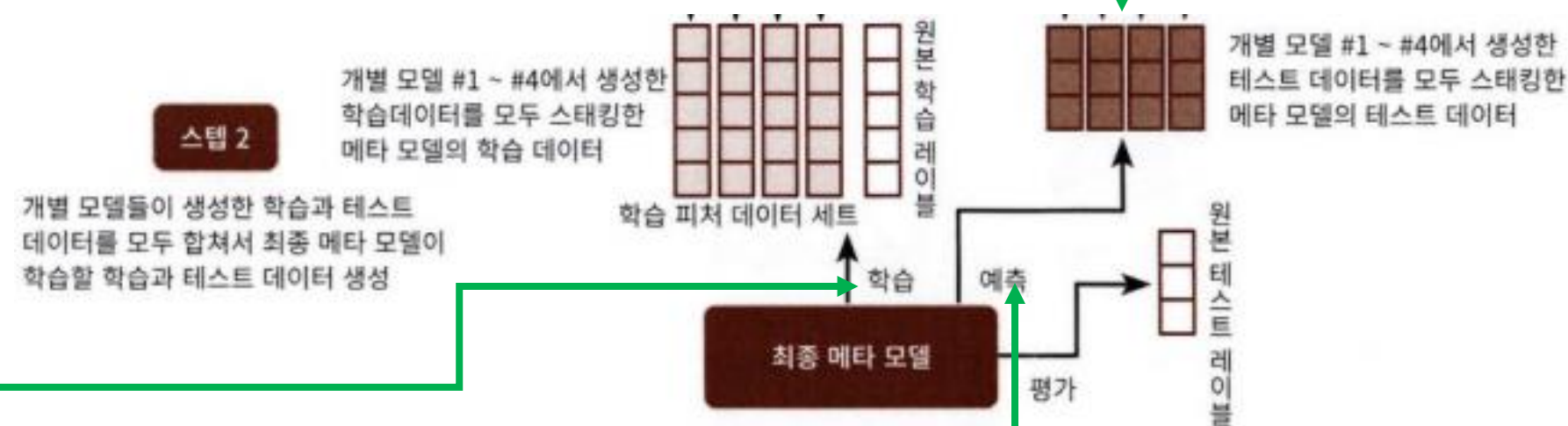
원본 학습 피쳐 데이터 Shape: (455, 30) 원본 테스트 피쳐 Shape: (114, 30)
스태킹 학습 피쳐 데이터 Shape: (455, 4) 스택킹 테스트 피쳐 데이터 Shape: (114, 4)

```
lr_final.fit(Stack_final_X_train, y_train)
stack_final = lr_final.predict(Stack_final_X_test)

print('최종 메타 모델의 예측 정확도: {0:,.4f}'.format(accuracy_score(y_test, stack_final)))
```

최종 메타 모델인 로지스틱 회귀를 스택킹된 학습용 피쳐 데이터 세트와 원본 학습 레이블 데이터로 학습한 후에 스택킹된 테스트 데이터 세트로 예측하고, 예측 결과를 원본 테스트 레이블 데이터와 비교해 정확도를 측정.

최종 메타 모델의 예측 정확도: 0.9825

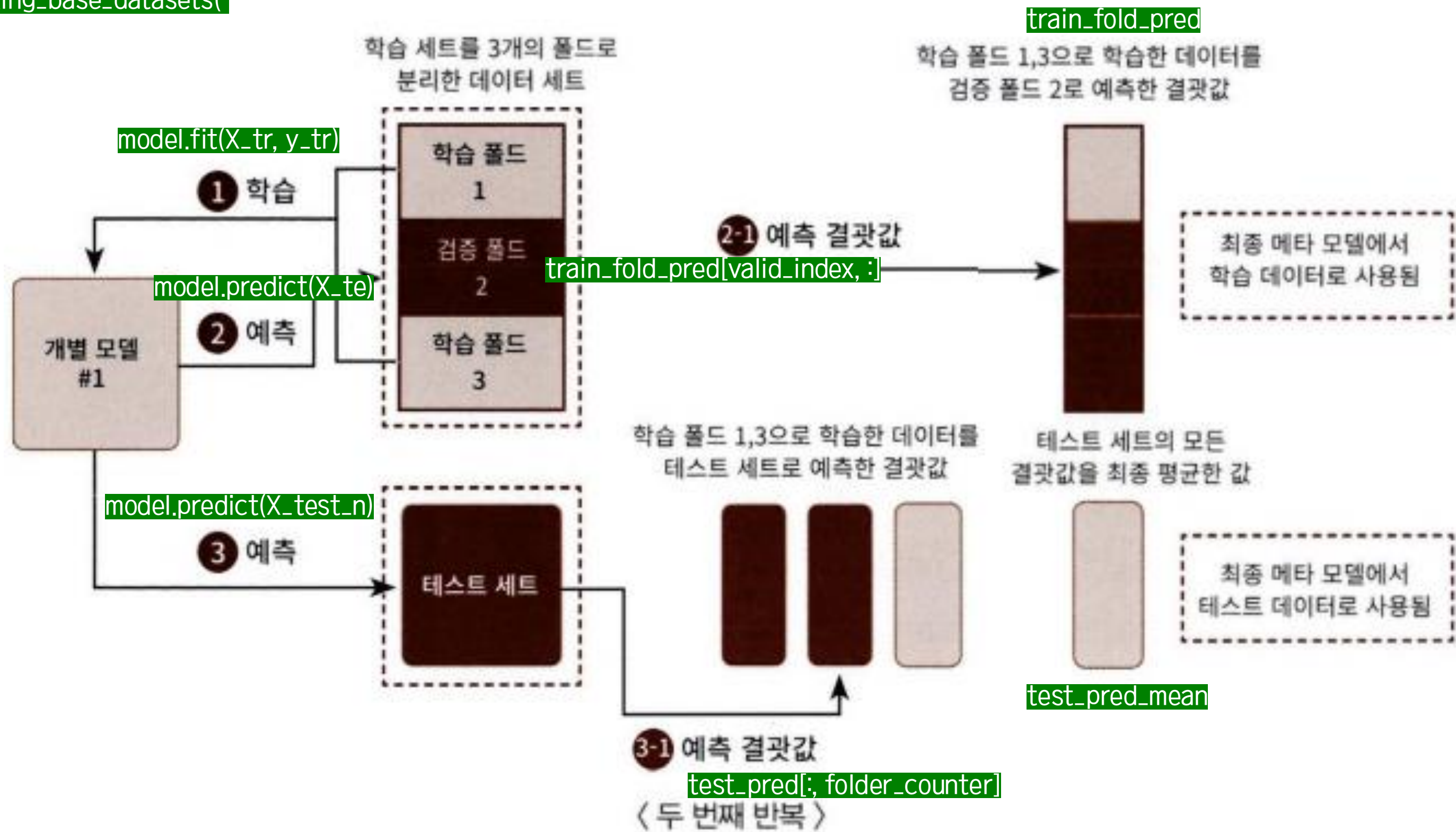


- 스택킹을 이루는 모델은 최적으로 파라미터를 튜닝(스태킹 모델의 파라미터 튜닝은 개별 알고리즘 모델의 파라미터를 최적으로 튜닝)한 상태에서 스택킹 모델을 만드는 것이 일반적.
- 스택킹 모델은 분류(Classification)뿐만 아니라 회귀(Regression)에도 적용 가능

#4.11 스택킹 앙상블

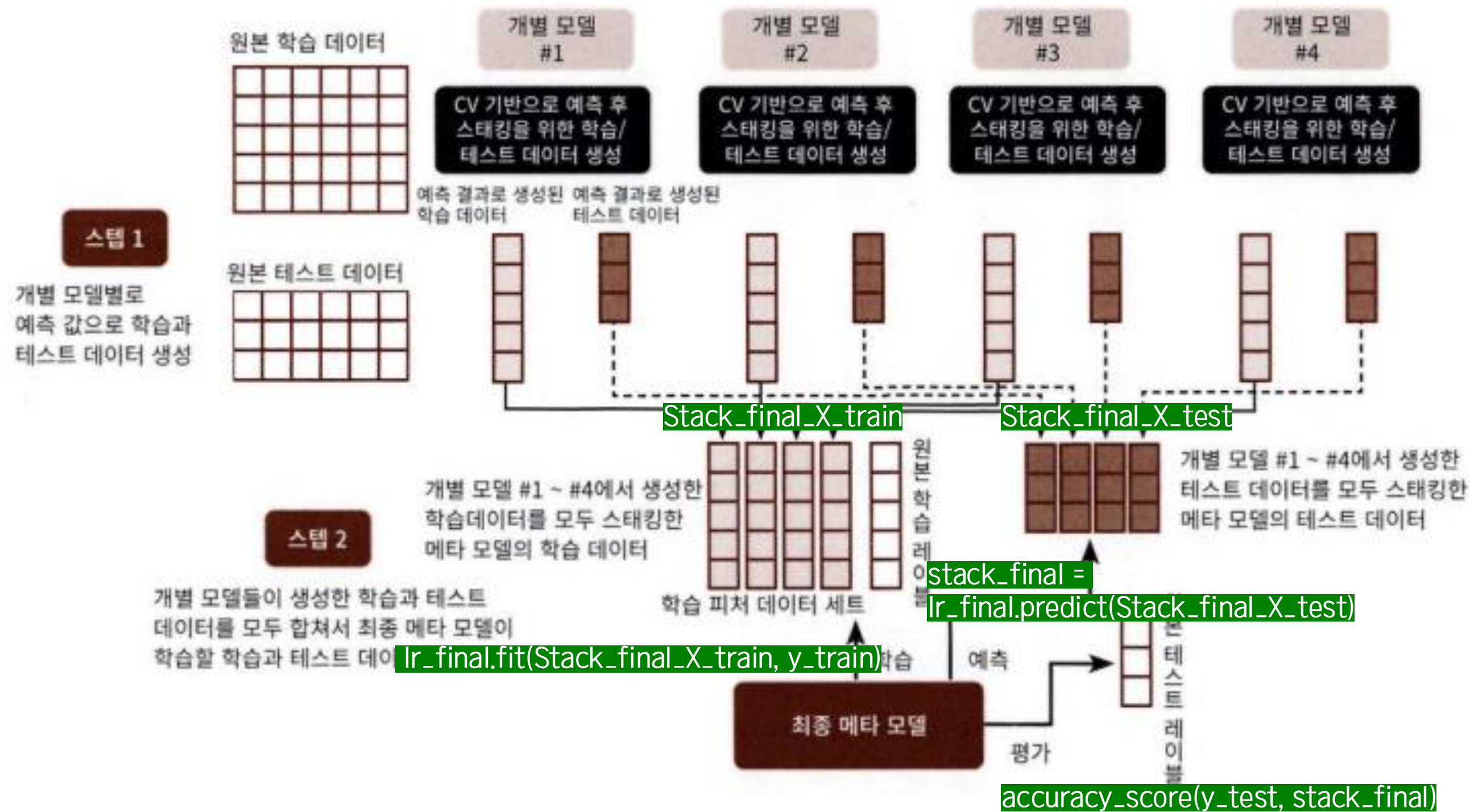
스텝1

```
get_stacking_base_datasets(  
    )
```



#4.11 스택킹 앙상블

스프2



THANK YOU

